HLD

SYSTEM DESIGN · CONCEPTS & BUILDING BLOCKS · SDE2 / SSE

High-Level Design

> ఈ document చదివిన తర్వాత System Design మళ్ళీ మర్చిపోలేవు. ప్రతి concept కి – **diagram**, real-life analogy, ఎప్పుడు/ఎందుకు వాడాలి, trade-offs, మరియు interview దృష్టి.

OOP → **LLD** → **HLD** అనే ప్రయాణంలో చివరి, అతి పెద్ద మెట్టు. లక్ష్యం: SDE2 మరియు SSE system design rounds ని confident గా clear చేయడం. Interview lo అడిగే *design problems* కోసం `HLD_Design_Problems_Telugu.pdf` చూడండి; ఇది వాటి వెనక ఉన్న *పరికరాల పెట్టె.*

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 1 – పునాదులు (Foundations)

1. HLD అంటే ఏమిటి? (HLD vs LLD, SDE2 vs SSE)
2. System Design Interview - ఎలా approach చేయాలి
3. Back-of-the-envelope Estimation (Capacity Planning)
4. Non-Functional Requirements (Scalability, Availability, ...)

Part 2 – Core Building Blocks 5. DNS, Client-Server, Networking basics 6. Load Balancing 7. Caching (patterns, eviction, invalidation) 8. CDN (Content Delivery Network) 9. SQL vs NoSQL 10. Database Replication 11. Sharding & Partitioning 12. Indexing

Part 3 – Distributed Systems 13. CAP Theorem + PACELC 14. Consistency Models 15. Consistent Hashing 16. Replication Strategies + Quorum 17. Message Queues & Kafka 18. Idempotency 19. Distributed Transactions (2PC, Saga) 20. Consensus & Leader Election (Raft) + Distributed Locks

Part 4 – Architecture & APIs 21. Monolith vs Microservices 22. API Gateway + Service Discovery 23. API Design (REST vs gRPC vs GraphQL) 24. Realtime (WebSockets, SSE, Long Polling) 25. Event-Driven Architecture (CQRS, Event Sourcing) 26. Resilience (Circuit Breaker, Retry, Bulkhead, Rate Limiting) 27. Search (Inverted Index) + Bloom Filters

Part 5 – Reliability & Operations 28. Availability (9s), SLA / SLO / SLI 29. Observability (Metrics, Logs, Traces) 30. Deployment (Blue-Green, Canary, Rolling) 31. Failover & Disaster Recovery

Part 6 – Case Studies (System Design) 32. URL Shortener (TinyURL) 33. Distributed Rate Limiter 34. News Feed (Twitter / Instagram) 35. Chat System (WhatsApp) 36. Video Streaming (YouTube / Netflix) 37. Notification System 38. Uber / Ride-sharing (Geo)

Part 7 – SSE Deep Dives 39. Storage Internals (LSM-tree vs B-tree) 40. Isolation Levels & Consistency Depth 41. Stream Processing (Batch vs Stream) 42. Multi-Region & CRDTs 43. Security in System Design 44. Cost, Load Shedding & Graceful Degradation

Part 8 – Advanced Case Studies 45. Payment System / Ledger 46. Collaborative Editing (Google Docs) 47. Typeahead / Autocomplete 48. Distributed Job Scheduler (Cron at Scale)

Part 9 – Interview & Reference 49. SDE2 vs SSE - ఏం expect చేస్తారు 50. HLD Interview Framework (Step-by-step) 51. Numbers & Latency Cheat Sheet 52. Memory Tips + Common Mistakes

---

Part 1 – పునాదులు (Foundations)

1. HLD అంటే ఏమిటి? (HLD vs LLD, SDE2 vs SSE)

వివరణ

HLD (High-Level Design) = మొత్తం system ని **పెద్ద బొమ్మ** స్థాయిలో design చేయడం - ఏ services, ఏ databases, ఏ caches, అవి ఎలా మాట్లాడుకుంటాయి, scale ఎలా అవుతాయి.

- **HLD** = "అడవి" (forest) - components, data flow, architecture.
- **LLD** = "చెట్టు" (tree) - ఒక్క component లోపల classes, methods.

Real-life Scenario

> ఒక **నగరం** ప్లాన్ చేస్తున్నామనుకో:
>
> - **HLD** = ఎక్కడ highways, ఎక్కడ water plants, power grids, ఏ ప్రాంతం residential vs industrial - city master plan.
> - **LLD** = ఒక్క ఇంటి లోపల room layout, wiring, plumbing.

HLD vs LLD

అంశం	HLD	LLD
Scope	మొత్తం system architecture	ఒక component లోపల classes
ప్రశ్న	ఏ services + ఎలా scale?	ఏ classes + ఏ methods?
Output	Architecture diagram, data flow	Class diagram, code
Focus	Scalability, availability, trade-offs	SOLID, patterns, clean code
ఉదా	"Feed service + Cache + Kafka + Sharded DB"	"Post, Feed, User classes + Observer"

SDE2 vs SSE - HLD లో తేడా

అంశం	SDE2 (mid-level)	SSE (senior)
Scope	ఇచ్చిన problem ని design చేయడం	Ambiguous problem ని scope చేయడం + design
Depth	Building blocks సరిగ్గా వాడటం	Deep trade-offs, failure modes, edge cases
Trade-offs	తెలుసు, వాడగలడు	ప్రతి నిర్ణయాన్ని *justify* చేస్తాడు (ఎందుకు ఇది, ఇంకోటి కాదు)

అంశం	SDE2 (mid-level)	SSE (senior)
Scale	Millions users	100M+ users, multi-region, bottlenecks
Estimation	చేయగలడు	Estimation తో design decisions drive చేస్తాడు
Ownership	Component design	End-to-end system + operational concerns (monitoring, DR)
Communication	స్పష్టంగా చెప్పగలడు	Interviewer ని lead చేస్తాడు, alternatives propose చేస్తాడు

> **కీలకం:** SDE2 "ఎలా build చేయాలి" చూపిస్తాడు. SSE "**ఎందుకు ఇలా**, ఇంకో విధంగా ఎందుకు కాదు, fail అయితే ఏమవుతుంది" అని *reasoning* చూపిస్తాడు. HLD interview లో **trade-off reasoning** = seniority signal.

---

2. System Design Interview - ఎలా approach చేయాలి

వివరణ

"Design YouTube" లాంటి ఓపెన్ ప్రశ్న ఇచ్చినప్పుడు, నేరుగా boxes గీయడం మొదలుపెట్టకూడదు. ఒక క్రమం (framework) follow చేయాలి. దీన్ని **RESHADED** లేదా simple 7-step గా గుర్తుంచుకో.

7 అడుగుల Framework

```
1. REQUIREMENTS (5 min)   - Functional + Non-functional. Scope narrow చేయి.
2. ESTIMATION (5 min)     - QPS, storage, bandwidth. (Part 1, Topic 3)
3. API DESIGN (5 min)     - ముఖ్య endpoints (REST/gRPC).
4. DATA MODEL (5 min)     - Entities, ఏ DB (SQL/NoSQL), schema.
5. HIGH-LEVEL DIAGRAM     - Boxes: client → LB → services → cache → DB.
6. DEEP DIVE (15 min)     - 1-2 components deep: scaling, sharding, bottlenecks.
7. WRAP UP (5 min)        - Bottlenecks, trade-offs, failure modes, monitoring.
```

ప్రతి అడుగులో ముఖ్య ప్రశ్నలు

అడుగు	అడగాల్సిన ప్రశ్నలు
Requirements	Users ఎంత? Read-heavy vs write-heavy? ఏ features MVP?
Estimation	DAU? QPS peak? Storage/year?
Data Model	Relations ఎక్కువా (SQL)? Scale ఎక్కువా (NoSQL)?
Deep Dive	Bottleneck ఎక్కడ? Single point of failure ఏది?

ముఖ్య సూత్రాలు

- **Requirements ముందు clarify చేయి** - assumptions చెప్పు (interviewer confirm చేస్తాడు)
- **Think aloud** - నీ ఆలోచన బయటికి చెప్పు (silence = negative)
- **Simple తో మొదలుపెట్టి scale చేయి** - ముందు single server, తర్వాత "ఇది scale అవ్వదు కాబట్టి..." అని evolve చేయి
- **Trade-offs మాట్లాడు** - "SQL వాడతాను ఎందుకంటే... కానీ scale కి NoSQL కూడా option"
- **Numbers తో justify చేయి** - "1M QPS కాబట్టి ఒక server సరిపోదు"

> **Golden rule:** Perfect design లేదు - *justified* design ఉంది. ప్రతి box కి "ఎందుకు?" అని సమాధానం ఉండాలి.

---

3. Back-of-the-envelope Estimation (Capacity Planning)

వివరణ

Design decisions ని **numbers** తో justify చేయాలి. "ఒక server సరిపోతుందా? ఎన్ని కావాలి? ఎంత storage?" - వీటికి ముతక (rough) లెక్కలు. Interviewer precision కాదు, *reasoning* చూస్తాడు.

గుర్తుంచుకోవాల్సిన Numbers

అంశం	విలువ (గుర్తుంచుకో)
1 day	~86,400 సెకన్లు (~10^5)
1 million writes/day	~12 writes/sec
1 char	1 byte (ASCII), 2-4 (Unicode)
KB → MB → GB → TB	ప్రతిది ×1000
1 million users, 1KB each	1 GB
SSD read	~100 μs; Network round-trip (same DC) ~0.5 ms

Estimation Steps (ఒక క్రమం)

```
1. DAU (Daily Active Users) — assume చేయి (ఉదా: 100M)
2. QPS = (DAU × actions/user) / 86400
3. Peak QPS = Average QPS × 2 నుండి 3 (spikes కి)
4. Storage = records/day × size/record × retention (years)
5. Bandwidth = QPS × payload size
6. Cache size = 20% hot data (80–20 rule)
```

ఉదాహరణ - Twitter లాంటి system

```
// Back-of-envelope calculator (interview లో మనసులో చేసేదాన్ని code గా)
const DAU = 100_000_000; // 100M daily active users
const tweetsPerUserPerDay = 2; // సగటున
const readWriteRatio = 100; // read-heavy (100 reads : 1 write)

const SECONDS_PER_DAY = 86_400;

// Write QPS
const writesPerDay = DAU * tweetsPerUserPerDay;
const writeQPS = Math.round(writesPerDay / SECONDS_PER_DAY);

// Read QPS
const readQPS = writeQPS * readWriteRatio;

// Peak (spikes కి ×3)
const peakReadQPS = readQPS * 3;

// Storage/year (ఒక్కో tweet ~300 bytes)
const bytesPerTweet = 300;
const storagePerYearGB = Math.round((writesPerDay * bytesPerTweet * 365) / 1e9);

console.log(`Write QPS:      ~${writeQPS.toLocaleString()}`); // ~2,315
console.log(`Read QPS:       ~${readQPS.toLocaleString()}`); // ~231,500
console.log(`Peak Read QPS:  ~${peakReadQPS.toLocaleString()}`); // ~694,500
console.log(`Storage/year:   ~${storagePerYearGB.toLocaleString()} GB`); // ~21,900
GB (~22 TB)
```

ఈ numbers ఎలా decisions ని drive చేస్తాయి

- **Read QPS 230K** → ఒక DB సరిపోదు → **read replicas + cache** అవసరం
- **Write QPS 2.3K** → single primary handle చేయవచ్చు (ఇప్పటికి)
- **22 TB/year** → **sharding** అవసరం (ఒక్క machine లో పట్టదు)
- **Read-heavy (100:1)** → aggressive caching justified

> **గుర్తుంచుకో:** Estimation యొక్క లక్ష్యం precision కాదు - ఏ *component ఎక్కడ break అవుతుందో* కనుక్కోవడం. Numbers మీ design ని guide చేస్తాయి.

---

4. Non-Functional Requirements (NFRs)

వివరణ

Functional requirements = system ఏం చేస్తుంది ("tweet post చేయాలి"). **Non-functional requirements** = system *ఎంత బాగా* చేస్తుంది (fast? reliable? scalable?). HLD లో అసలు కష్టం NFRs లోనే.

ముఖ్య NFRs

NFR	అర్థం	ఎలా కొలుస్తారు
Scalability	Load పెరిగినా handle చేయడం	Users/QPS పెరిగినా perf స్థిరం
Availability	System up ఉండటం	Uptime % (99.9%, 99.99%)
Reliability	సరైన ఫలితం, data పోకుండా	Failure rate, data loss
Latency	ఒక request కి పట్టే సమయం	p50, p95, p99 (milliseconds)
Throughput	యూనిట్ టైమ్‌లో requests	QPS / RPS
Consistency	అందరికీ ఒకే data కనిపించడం	Strong / Eventual
Durability	Save అయిన data శాశ్వతం	Data loss probability

Vertical vs Horizontal Scaling

	Vertical (scale up)	Horizontal (scale out)
ఏం చేస్తాం	ఒకే machine ని పెద్దది చేయడం (RAM, CPU)	ఎక్కువ machines జోడించడం
Limit	Hardware limit ఉంది	~Unlimited
Complexity	సులభం	Load balancer, distributed state
Failure	Single point of failure	Fault tolerant
ఉదా	DB server ని upgrade	10 app servers behind LB

> **నియమం:** Stateless services → horizontal easy. Stateful (DB) → sharding/replication అవసరం.

Latency Percentiles (p50, p95, p99)

> **p99 = 200ms** అంటే 99% requests 200ms లోపు, 1% నెమ్మది. Average మోసం చేస్తుంది - **tail latency (p99)** ముఖ్యం. 1M requests లో 1% = 10,000 unhappy users!

Latency vs Throughput

> - **Latency** = ఒక్క car ఒక చోటికి చేరే సమయం.
> - **Throughput** = గంటకి highway మీద ఎన్ని cars వెళ్తాయి. రెండూ వేరు - lanes పెంచితే (throughput↑) కానీ ఒక్క car వేగం (latency) అలాగే ఉండవచ్చు.

Key Points

- Functional "ఏం" చెప్తుంది; NFR "ఎంత బాగా" - HLD లో NFRs మీదే focus
- Everything is a trade-off: consistency ↔ availability, latency ↔ durability
- SSE level: ప్రతి NFR ని numbers తో target చేయి ("p99 < 200ms, 99.99% uptime")

Part 2 – Core Building Blocks

> ఏ పెద్ద system అయినా ఇవే blocks తో కడతారు: DNS, Load Balancer, Cache, CDN, Database (+ scaling). వీటిని బాగా అర్థం చేసుకుంటే, ఏ design అయినా కట్టవచ్చు.

---

5. DNS, Client-Server, Networking Basics

వివరణ

Browser లో `youtube.com` type చేస్తే ఏం జరుగుతుంది? ఈ flow HLD కి పునాది.

Request Flow (URL → Response)

```
1. DNS Lookup    : youtube.com → IP address (142.x.x.x)
2. TCP Handshake : client ↔ server connection (3-way)
3. TLS Handshake : HTTPS encryption setup
4. HTTP Request  : GET /watch?v=...
5. Server        : Load Balancer → App Server → DB/Cache
6. HTTP Response : HTML/JSON తిరిగి
```

DNS అంటే

> **DNS = ఇంటర్నెట్ యొక్క phone book.** పేరు (youtube.com) → నంబర్ (IP). నువ్వు friend పేరు గుర్తుంచుకుంటావు, phone నంబర్ కాదు - DNS అదే చేస్తుంది domains కి.

- DNS కూడా **load balancing** చేస్తుంది (ఒకే domain → అనేక IPs, geo-based routing)
- DNS responses **cache** అవుతాయి (TTL) - వేగం కోసం

Protocols (త్వరగా)

Protocol	వాడకం
HTTP/HTTPS	Web requests (request-response)
TCP	Reliable, ordered (ఫైళ్ళు, web)
UDP	Fast, unreliable (video, games, DNS)
WebSocket	Full-duplex (chat, realtime)

Key Points

- HLD diagram లో ఎప్పుడూ client → DNS → LB → services flow ఉంటుంది

- Same-datacenter round-trip ~0.5ms; cross-continent ~150ms (light speed limit!)
- Latency తగ్గించడానికి: user కి దగ్గరగా (CDN, edge, multi-region)

---

6. Load Balancing

వివరణ

Load Balancer (LB) = incoming requests ని అనేక servers కి పంచే traffic police. ఒక server మీద భారం పడకుండా, fault tolerance కి కీలకం.

Real-life Scenario

> **Bank లో ఒకే queue, అనేక counters.** Manager (LB) "మీరు counter 3 కి వెళ్ళండి" అని పంపుతాడు - ఏ counter ఖాళీగా ఉందో చూసి. ఒక counter మూసేసినా, మిగతావి పని చేస్తాయి.

LOAD BALANCING · ఎక్కడ, ఎలా పంచాలి

L4 vs L7 – ఎక్కడ నిర్ణయం జరుగుతుంది

L4 (Transport)
IP + port మాత్రమే చూస్తుంది · content చూడదు · చాలా వేగం

L7 (Application)
URL, header, cookie చూస్తుంది · /api → A, /img → B · తెలివైనది

ALGORITHMS

Round Robin
వరుసగా ఒక్కొక్కరికి

Least Connections
తక్కువ busy server కి

Weighted
పెద్ద server కి ఎక్కువ

IP Hash
ఒకే client → ఒకే server

Health check – ఇదే load balancer యొక్క అసలు విలువ
ప్రతి కొన్ని సెకన్లకి /health కొట్టి, జవాబు ఇవ్వని server ని pool నుంచి తీసేయడం. Traffic పంచడం కంటే ఇదే ముఖ్యం.

Sticky session ఉచ్చు: IP hash తో ఒకే user ఎప్పుడూ ఒకే server కి వెళ్తాడు – session memory lo ఉంచొచ్చు. కానీ ఆ server చస్తే session పోతుంది, మరియు scaling అసమానం అవుతుంది. మేలైనది: **servers ని stateless గా ఉంచి session ని Redis lo** పెట్టడం.

L4 vs L7 Load Balancer

	L4 (Transport)	L7 (Application)
ఏ ఆధారంగా	IP + Port	HTTP content (URL, headers, cookies)
వేగం	ఎక్కువ (simple)	తక్కువ (content చదవాలి)
Smart routing	కుదరదు	`/api` → service A, `/video` → service B
ఉదా	TCP-level	Nginx, HAProxy, ALB

Load Balancing Algorithms

Algorithm	ఎలా పంచుతుంది
Round Robin	వరుసగా ఒక్కొక్కరికి
Weighted RR	పెద్ద servers కి ఎక్కువ
Least Connections	తక్కువ busy server కి
IP Hash	ఒకే client ఎప్పుడూ ఒకే server కి (sticky)
Consistent Hashing	Server add/remove అయినా minimal remapping (Topic 15)

Health Checks

> LB ప్రతి server ని ping చేస్తూ ఉంటుంది. Server fail అయితే, దాన్ని rotation నుండి తీసేస్తుంది (requests పంపదు). Server తిరిగి healthy అయితే, మళ్ళీ చేరుస్తుంది.

Key Points

- LB = horizontal scaling + high availability కి కీలకం
- LB కూడా single point of failure కాకూడదు → **active-passive LB pair** (2 LBs)
- Stateless servers అయితే LB easy; session state అయితే sticky sessions లేదా external store

---

7. Caching

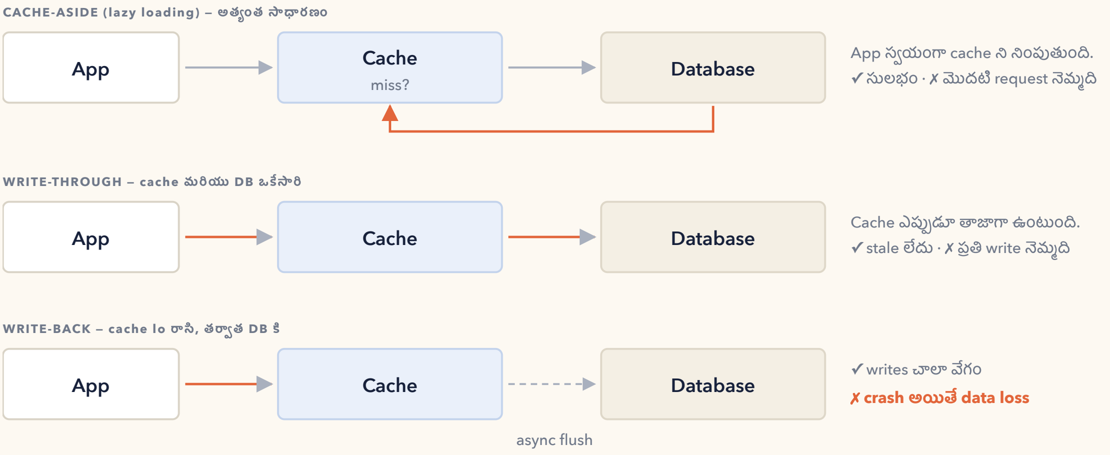
వివరణ

Cache = తరచూ కావలసిన data ని వేగవంతమైన storage (RAM) లో ఉంచడం. DB కి వెళ్ళే భారం తగ్గించి, latency తగ్గిస్తుంది. **Read-heavy systems కి అతి ముఖ్యం.**

Real-life Scenario

> **Fridge vs Market.** ప్రతిసారి పాలు కోసం market కి వెళ్ళవు (DB) - fridge లో (cache) ఉంచుకుంటావు. వేగం. కానీ fridge చిన్నది (RAM ఖరీదు), పాలు పాడవుతాయి (stale data) - అందుకే expiry (TTL).

మూడు CACHING PATTERNS · ఎప్పుడు ఏది



నియమం: Read-heavy + కొంత staleness ఫర్వాలేదు → cache-aside. Stale అస్సలు వద్దు → write-through. Write-heavy మరియు కొంత data loss భరించగలరు (analytics counters) → write-back.

ఎక్కడ cache చేయవచ్చు

```
Client (browser cache) → CDN → Load Balancer →
   App Server (local/in-memory) → Distributed Cache (Redis) → Database
```

Caching Patterns

Pattern	ఎలా పనిచేస్తుంది	ఎప్పుడు
Cache-Aside (Lazy)	App: cache లో లేకపోతే DB నుండి తెచ్చి cache లో పెట్టు	అత్యంత common (read-heavy)
Read-Through	Cache తనే DB నుండి load చేస్తుంది	Cache library handle చేస్తే
Write-Through	DB + cache రెండిటికీ ఒకేసారి write	Consistency ముఖ్యం అయితే
Write-Back	Cache కి write, తర్వాత DB కి async	Write-heavy, కానీ data loss risk

Cache Eviction Policies

Policy	ఏది తీసేస్తుంది
LRU	చాలా కాలం వాడనిది (అత్యంత common - LLD Topic 34)
LFU	అతి తక్కువ సార్లు వాడినది
FIFO	మొదట వచ్చినది మొదట
TTL	సమయం అయిపోయినది

Cache Invalidation (అతి కష్టమైన సమస్య)

> "Computer science లో 2 కష్ట సమస్యలు: cache invalidation, naming things." - Stale data ని ఎప్పుడు తీసేయాలో నిర్ణయించడం కష్టం.

- **TTL** - సమయం అయిపోయాక auto-expire (simple కానీ stale window ఉంటుంది)
- **Write-through invalidation** - data మారినప్పుడు cache update/delete
- **Event-based** - DB change event → cache invalidate

ముఖ్య సమస్యలు (SSE level)

సమస్య	అర్థం	పరిష్కారం
Cache Stampede	Key expire అయ్యే క్షణం వేలాది requests DB కి	Lock/single-flight, jittered TTL
Hot Key	ఒకే key కి భారీ traffic	Replicate hot key, local cache
Thundering Herd	Cache empty → అందరూ DB కి	Pre-warm, request coalescing

Key Points

- Cache hit ratio ముఖ్యం (95%+ మంచిది) - hit ratio తక్కువైతే cache వృథా
- 80-20 rule: 20% hot data 80% requests - దాన్ని cache చేయి
- Cache = performance, కానీ consistency trade-off (stale data risk)

---

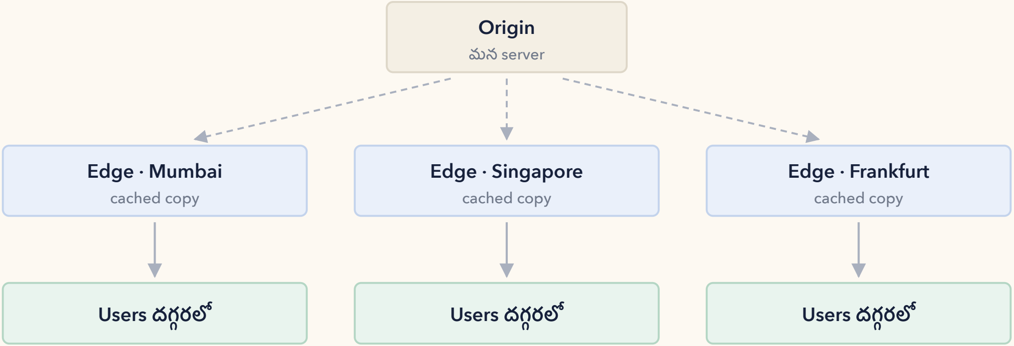
8. CDN (Content Delivery Network)

వివరణ

CDN = static content (images, videos, CSS, JS) ని ప్రపంచవ్యాప్తంగా **edge servers** లో copy చేసి, user కి దగ్గరి server నుండి serve చేయడం. Latency బాగా తగ్గిస్తుంది.

Real-life Scenario

> **Amazon warehouses.** ఒకే central warehouse నుండి కాకుండా, ప్రతి నగరంలో warehouse ఉంటే delivery వేగంగా. CDN అలా content ని user దగ్గరికి తెస్తుంది.



PULL (lazy)	PUSH (pre-warm)
మొదటి user origin నుంచి తెస్తాడు, తర్వాత cache	ముఖ్యమైన content ని ముందే edges కి పంపడం

CDN కేవలం వేగం కోసం కాదు – ఇది **origin ని కాపాడుతుంది**. Cache hit ratio 90% నుంచి 95% కి పెరిగితే origin traffic **సగం** అవుతుంది. Video/image-heavy products lo ఇదే అతి పెద్ద ఖర్చు తగ్గింపు.

ఎలా పనిచేస్తుంది

```
User (Hyderabad) → CDN edge (Hyderabad) : content ఉంటే వెంటనే (cache hit)
                                         : లేకపోతే origin నుండి తెచ్చి cache (cache miss)
User (London)    → CDN edge (London)     : వాళ్ళ దగ్గర copy
```

Push vs Pull CDN

	Pull	Push
Content ఎలా వస్తుంది	User మొదటిసారి అడిగినప్పుడు origin నుండి pull	మనమే ముందే upload
ఎప్పుడు	సాధారణ websites (auto)	పెద్ద files, ముందే తెలిస్తే

Key Points

- Static content (images, video, JS/CSS) → CDN; dynamic content → origin
- CDN latency బాగా తగ్గిస్తుంది + origin మీద భారం తగ్గిస్తుంది
- Cloudflare, Akamai, CloudFront - popular CDNs
- Video streaming, global websites కి CDN తప్పనిసరి

---

9. SQL vs NoSQL

వివరణ

Database ఎంపిక HLD లో అతి ముఖ్య నిర్ణయం. **SQL** (relational) vs **NoSQL** (non-relational) - ప్రతిదానికీ దాని స్థానం.

పోలిక

అంశం	SQL (MySQL, Postgres)	NoSQL (Mongo, Cassandra, DynamoDB)
Schema	Fixed (rows/columns)	Flexible (documents, key-value)
Relations	Joins, foreign keys	పరిమితం (denormalize చేస్తాం)
Transactions	ACID (strong)	తరచూ eventual (BASE)
Scaling	Vertical (sharding కష్టం)	Horizontal (built-in)
ఎప్పుడు	Complex queries, relations, consistency	Massive scale, flexible schema
ఉదా	Banking, orders, users	Feeds, logs, IoT, catalogs

NoSQL రకాలు

రకం	Structure	ఉదా	వాడకం
Key-Value	key → value	Redis, DynamoDB	Cache, sessions
Document	JSON documents	MongoDB	Catalogs, profiles
Column-family	Wide columns	Cassandra, HBase	Time-series, write-heavy
Graph	Nodes + edges	Neo4j	Social networks, recommendations

ACID vs BASE

ACID (SQL)	BASE (NoSQL)
Atomicity	**BA**sically **A**vailable
Consistency	**S**oft state
Isolation	**E**ventual consistency
Durability	
Strong guarantees	High availability, scale

ఎప్పుడు ఏది? (Decision)

- **SQL వాడు:** strong consistency (money!), complex relations/joins, transactions, moderate scale
- **NoSQL వాడు:** massive scale, flexible/evolving schema, high write throughput, simple queries
- **రెండూ వాడు (Polyglot):** users → SQL, feed/logs → NoSQL, cache → Redis, search → Elasticsearch

> **SSE tip:** "NoSQL = scale" అని గుడ్డిగా చెప్పకు. Modern SQL (Postgres, Vitess, CockroachDB) కూడా బాగా scale అవుతుంది. **Access pattern** బట్టి నిర్ణయించు.

---

10. Database Replication

వివరణ

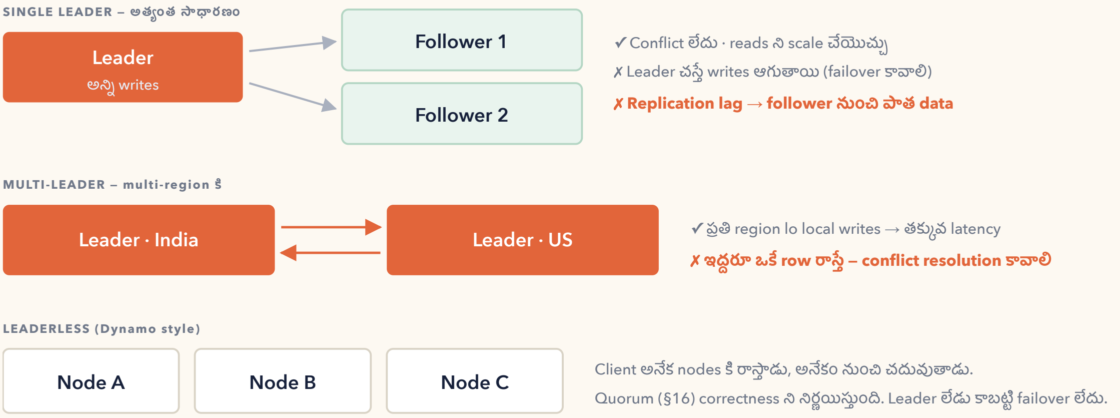
Replication = data ని అనేక DB copies లో ఉంచడం. High availability (ఒకటి fail అయినా), read scaling (reads ని పంచడం) కోసం.

Leader-Follower (Master-Slave)

```
      Writes                  Reads
        │                   ┌───┴───┐
        ▼                   ▼       ▼
   ┌──────────┐  replicate ┌────────┐ ┌────────┐
   │  Leader  │───────────>│Follower│ │Follower│
   │(Primary) │            │(Replica)│ │(Replica)│
   └──────────┘            └────────┘ └────────┘
```

- **Writes** → Leader కి మాత్రమే
- **Reads** → Followers నుండి (read scaling!)
- Leader fail → ఒక follower ని leader గా promote (failover)

REPLICATION · మూడు TOPOLOGIES



Replication lag ప్రతి approach lo ఉంటుంది. "నేను profile update చేశాను కానీ refresh చేస్తే పాతది కనిపిస్తోంది" – ఇదే. పరిష్కారం: **read-your-writes consistency** – ఆ user reads ని కొంతసేపు leader నుంచే ఇవ్వడం.

Sync vs Async Replication

	Synchronous	Asynchronous
Write confirm	Followers copy అయ్యాకే	Leader write అయ్యాకే వెంటనే

	Synchronous	Asynchronous
Consistency	Strong (data loss లేదు)	Follower వెనుకబడవచ్చు (lag)
Latency	ఎక్కువ (wait)	తక్కువ (fast)
Risk	Follower slow → write slow	Leader crash → కొంత data loss

Replication రకాలు

రకం	వివరణ
Single-Leader	ఒక leader, అనేక followers (common)
Multi-Leader	అనేక leaders (multi-region writes) - conflict resolution అవసరం
Leaderless	ఏ node కైనా write (Cassandra, Dynamo) - quorum (Topic 16)

Replication Lag సమస్య

> Async replication లో follower వెనుకబడితే - user తన సొంత write ని వెంటనే చదవలేకపోవచ్చు ("నేను post చేశా కానీ కనిపించలేదు!"). పరిష్కారం: **read-your-own-writes** (తన writes ని leader నుండి చదవడం).

Key Points

- Replication = availability + read scaling; కానీ writes ని scale చేయదు (అది sharding)
- Async = fast కానీ eventual; Sync = consistent కానీ slow
- Leader failover కి automatic detection + promotion అవసరం

---

11. Sharding & Partitioning

వివరణ

Sharding = ఒక పెద్ద DB ని అనేక చిన్న DBs (shards) గా విభజించడం, ఒక్కో shard లో data లో ఒక భాగం. **Write scaling** కి ఏకైక మార్గం (replication reads మాత్రమే scale చేస్తుంది).

Real-life Scenario

> **ఒక పెద్ద library ని అనేక branches గా విభజించడం.** A-M పుస్తకాలు branch 1, N-Z branch 2. ఏ పుస్తకం ఎక్కడుందో తెలిస్తే నేరుగా అక్కడికి వెళ్తావు. ఒక branch నిండినా, మిగతావి ఖాళీ.

SHARDING · డేటాను ఎలా విభజించాలి

RANGE – key విలువ ప్రకారం

A - H	I - P	Q - Z
Shard 1	Shard 2	Shard 3

✓ Range queries సులభం ("A తో మొదలయ్యే అందరూ")

✗ Hotspot – "S" పేర్లు ఎక్కువ ఉంటే shard 3 మునిగిపోతుంది

HASH – hash(key) % N

hash → 0	hash → 1	hash → 2
Shard 1	Shard 2	Shard 3

✓ సమానంగా పంపిణీ, hotspot అరుదు

✗ Range queries సాధ్యం కాదు · N మారితే అన్నీ కదులుతాయి → §15

DIRECTORY – ఒక lookup table

Lookup service
key → ఏ shard lo ఉందో చెప్తుంది
→ Shard 1 | Shard 2 | Shard 3

✓ అత్యంత flexible – ఏ key నైనా ఎక్కడికైనా తరలించొచ్చు · ✗ lookup service ఒక SPOF అవుతుంది

అసలు కష్టం shard key ఎంచుకోవడం. తప్పుగా ఎంచుకుంటే – hotspot, లేదా ప్రతి query అన్ని shards ని అడగాల్సి రావడం (scatter-gather). మంచి shard key: *అధిక cardinality + query pattern కి సరిపోయేది.*

Sharding Strategies

Strategy	ఎలా	సమస్య
Range-based	A-M shard1, N-Z shard2	Hotspots (కొన్ని ranges busy)
Hash-based	hash(key) % N	Even distribution కానీ range queries కష్టం
Consistent Hashing	hash ring (Topic 15)	Node add/remove smooth
Geo-based	Region బట్టి	Latency తక్కువ, కానీ uneven

Sharding సవాళ్ళు (SSE level)

సవాలు	వివరణ
Cross-shard queries	అనేక shards నుండి join కష్టం/ఖరీదు
Hotspots	ఒక shard కి ఎక్కువ traffic (celebrity user)
Resharding	Shards పెంచేటప్పుడు data move (consistent hashing సహాయం)
Distributed transactions	Shards మధ్య transaction కష్టం (Saga - Topic 19)

Partitioning vs Sharding

> **Partitioning** = ఒకే DB లోపల tables ని విభజించడం (logical). **Sharding** = అనేక machines మీద పంచడం (physical). Sharding = horizontal partitioning across servers.

Shard Key ఎంపిక (అతి ముఖ్యం)

- మంచి shard key = **even distribution** + **queries ఒక shard లోనే** పూర్తయ్యేలా

- ఉదా: chat app → `shard by chat_id` (ఒక chat ఒక shard లో); బాడ్ = `shard by date` (ఈరోజు shard hot)

Key Points

- Sharding = write scaling; replication = read scaling + availability - **రెండూ కలిపి** వాడతారు
- Shard key ఎంపిక critical (మార్చడం చాలా కష్టం తర్వాత)
- Cross-shard operations ని వీలైనంత తగ్గించు

---

12. Indexing

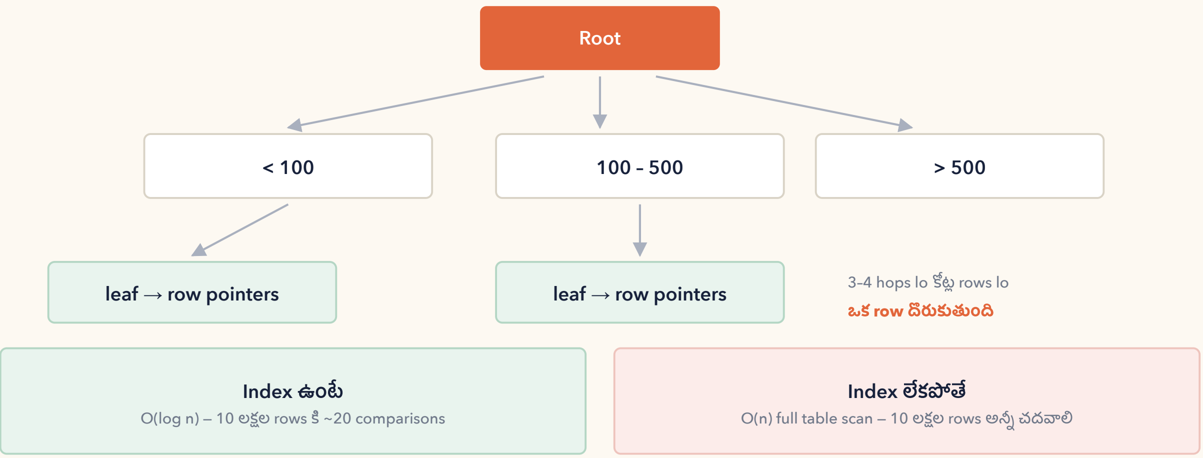
వివరణ

Index = DB లో వేగంగా వెతకడానికి ఒక data structure (సాధారణంగా B-Tree). Index లేకపోతే DB ప్రతి row చదవాలి (full scan).

Real-life Scenario

> **పుస్తకం చివర index.** ఒక topic కోసం మొత్తం పుస్తకం చదవవు - index చూసి నేరుగా page కి వెళ్తావు. DB index అలానే rows ని వేగంగా కనుక్కుంటుంది.

INDEXING · B-TREE ఎలా వెతుకుతుంది



కానీ ప్రతి index ఒక ఖరీదు: ప్రతి INSERT/UPDATE కి index ని కూడా update చేయాలి + disk space

Composite index lo క్రమం ముఖ్యం. `(city, age)` index – "city = హైదరాబాద్" queries కి పని చేస్తుంది, "age = 25" మాత్రమే ఉన్న query కి పని చేయదు. ఇది ఒక టెలిఫోన్ డైరెక్టరీ లాంటిది: ఇంటిపేరు తెలిస్తే వేగం, మొదటి పేరు మాత్రమే తెలిస్తే వృథా.

Trade-off

	Index తో	Index లేకుండా
Reads	వేగం (O(log n))	నెమ్మది (O(n) scan)
Writes	నెమ్మది (index update చేయాలి)	వేగం

	Index తో	Index లేకుండా
Storage	ఎక్కువ (index కి space)	తక్కువ

Index రకాలు

రకం	వాడకం
B-Tree	Range queries, sorting (default)
Hash	Exact match (=) మాత్రమే
Composite	అనేక columns కలిపి (`(last_name, first_name)`)
Inverted Index	Full-text search (Topic 27)

Key Points

- Read-heavy columns (WHERE, JOIN, ORDER BY) మీద index పెట్టు
- అతిగా index వద్దు - ప్రతి index writes ని నెమ్మది చేస్తుంది + space తింటుంది
- Index = read speed vs write speed + storage trade-off

Part 3 – Distributed Systems

> ఇక్కడే SDE2 నుండి SSE వేరు అవుతాడు. Distributed systems concepts - CAP, consistency, consistent hashing, quorum, Kafka, saga, consensus - వీటి మీద deep understanding = seniority.

---

13. CAP Theorem + PACELC

వివరణ

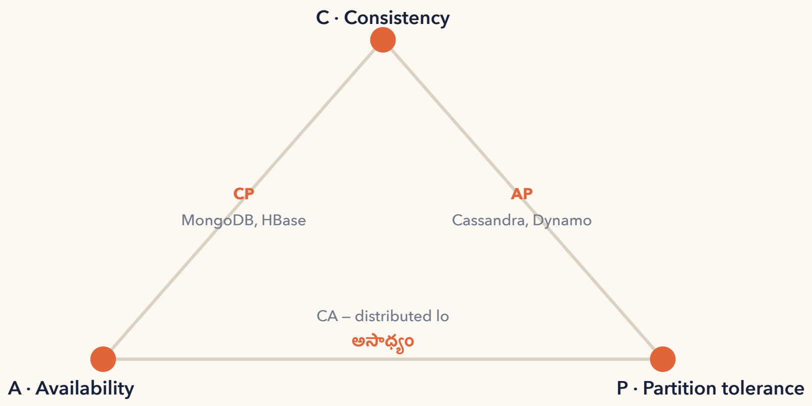
CAP Theorem: ఒక distributed system లో, network **P**artition (nodes మధ్య communication తెగడం) జరిగినప్పుడు, **C**onsistency మరియు **A**vailability - ఈ రెండిటిలో ఒకటే ఎంచుకోగలవు, రెండూ కాదు.

- **C**onsistency = అందరికీ ఒకే (latest) data
- **A**vailability = ప్రతి request కి సమాధానం (పాతదైనా)
- **P**artition tolerance = network తెగినా system పనిచేయడం

Real-life Scenario

> ఇద్దరు cashiers మధ్య phone line తెగింది (partition). ఒక account balance గురించి:
>
> - **CP:** "line వచ్చేదాకా ఆగండి" (consistent కానీ unavailable)
> - **AP:** "ఇప్పటి బ్యాలెన్స్ (పాతది కావచ్చు) ఇస్తా" (available కానీ inconsistent)

CAP THEOREM · PARTITION జరిగినప్పుడు ఏం వదులుకుంటారు



నిజం ఏమిటంటే – CAP ఒక "మూడింటిలో రెండు" ఎంపిక కాదు

Network partition జరగడం మన చేతిలో లేదు, అది జరిగే తీరుతుంది. కాబట్టి P తప్పనిసరి.
మిగిలిన ఏకైక ప్రశ్న: partition జరిగినప్పుడు C ని వదులుకుంటామా, A ని వదులుకుంటామా?

PACELC ఇంకా నిజాయితీగా చెప్తుంది: Partition ఉంటే A vs C; **Else** (సాధారణ సమయంలో) Latency vs Consistency. అంటే partition లేకపోయినా trade-off ఉంటూనే ఉంటుంది – sync replication = consistent కానీ నెమ్మది.

Partition ఉన్నప్పుడు ఎంపిక

ఎంపిక	అర్థం	ఉదా
CP (Consistency)	Partition లో inconsistent కంటే unavailable మేలు	Banking, MongoDB, HBase, Zookeeper
AP (Availability)	Partition లో stale అయినా అందుబాటు మేలు	Social feed, Cassandra, DynamoDB

> **ముఖ్యం:** Partition లేనప్పుడు C **మరియు** A రెండూ పొందవచ్చు. CAP కేవలం *partition సమయంలో* trade-off.

PACELC (CAP కి extension)

> **PACELC:** Partition (**P**) అయితే **A** vs **C**; **E**lse (normal) అయితే **L**atency vs **C**onsistency.
>
> అంటే partition లేకపోయినా, **latency** ↔ **consistency** trade-off ఎప్పుడూ ఉంటుంది (sync replication = consistent కానీ slow).

Key Points

- "CA system" అనేది distributed లో అసాధ్యం (partition ఎప్పుడూ జరగవచ్చు) → P తప్పనిసరి
- CP vs AP = business నిర్ణయం (money → CP, likes → AP)
- SSE: PACELC తో "normal లో కూడా latency vs consistency ఎంచుకుంటున్నా" అని చూపించు

---

14. Consistency Models

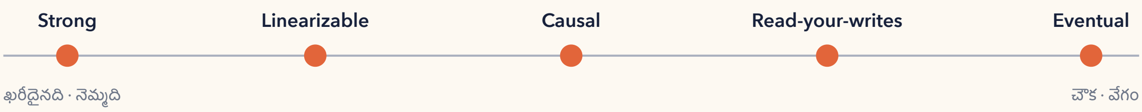
వివరణ

"Consistency" లో అనేక స్థాయిలు ఉన్నాయి - strong నుండి eventual వరకు. ఏ స్థాయి కావాలో business నిర్ణయిస్తుంది.

స్థాయిలు (strong → weak)

Model	అర్థం	ఉదా
Strong	Write తర్వాత అందరూ వెంటనే latest చూస్తారు	Banking balance
Linearizable	Strong + real-time order guarantee	Distributed locks
Causal	Cause-effect order కాపాడబడుతుంది	Comments (reply reply తర్వాతే)
Read-your-writes	నీ సొంత writes నీకు వెంటనే కనిపిస్తాయి	Profile edit
Eventual	కొంత సమయం తర్వాత అందరూ converge అవుతారు	Likes count, DNS

CONSISTENCY MODELS · బలమైనది నుంచి బలహీనమైనది వరకు



ఎక్కడ STRONG కావాలి	ఎక్కడ EVENTUAL చాలు
Account balance · inventory · seat booking · payment	Likes count · view count · DNS · social feed · analytics

ఒకే system lo రెండూ ఉండొచ్చు – Amazon lo cart eventual, checkout strong

సరైన ప్రశ్న: "ఇక్కడ ఎంత పాత data భరించగలం?" – "consistency కావాలా?" కాదు

Real-life Scenario

> **Eventual consistency:** నువ్వు photo like చేస్తే, నీకు వెంటనే 101 likes కనిపిస్తుంది, కానీ friend కి కొన్ని seconds పాటు 100 కనిపించవచ్చు. చివరికి ఇద్దరికీ 101. Likes కి ఇది ఓకే - **bank balance కి కాదు**.

Key Points

- Strong consistency = ఖరీదు (latency, availability trade-off)
- చాలా features కి eventual consistency సరిపోతుంది (likes, views, feeds)
- Money, inventory, bookings → strong; social signals → eventual
- SSE: "ఈ feature కి ఏ consistency level సరిపోతుంది?" అని ప్రతి దానికీ ఆలోచించు

---

15. Consistent Hashing

వివరణ

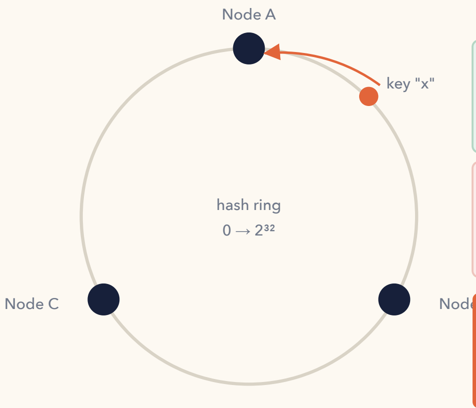
సమస్య: N servers మీద `hash(key) % N` తో data పంచితే - ఒక server add/remove అయితే **N మారుతుంది** → దాదాపు అన్ని keys remap అవుతాయి → cache/DB అంతా చెల్లాచెదురు.

పరిష్కారం - Consistent Hashing: servers మరియు keys ని ఒక **వృత్తం (hash ring)** మీద పెట్టడం. ఒక్కో key దాని clockwise లోని తర్వాతి server కి వెళ్తుంది. Server add/remove అయితే **కేవలం పక్క keys మాత్రమే** move అవుతాయి.

Real-life Scenario

> **గుండ్రటి బల్ల చుట్టూ కూర్చున్న అతిథులు.** కొత్తవాడు వస్తే, పక్కవాడి భారంలో కొంతే తీసుకుంటాడు - మొత్తం arrangement మారదు. `%N` అయితే ప్రతి కొత్తవాడికీ అందరూ కుర్చీలు మారాలి.

CONSISTENT HASHING · NODE చేరినా, పోయినా 1/N KEYS మాత్రమే కదులుతాయి



నియమం

Key ని hash చేసి ring మీద పెట్టు. అక్కడి నుంచి సవ్య దిశలో మొదట కనిపించే node దాని యజమాని.

సాధారణ hash తో పోలిస్తే

hash(key) % N – N మారితే దాదాపు అన్ని keys కదులుతాయి. ఒక node చేర్చడానికి మొత్తం cluster ని rebalance చేయాలి.

Virtual nodes ఎందుకు తప్పనిసరి

ఒక్కో physical node ని ring మీద 100+ చోట్ల పెట్టడం. లేకపోతే పంపిణీ అసమానం, మరియు ఒక node పోతే దాని భారం మొత్తం ఒక్క పొరుగువాడి మీద పడుతుంది.

Code

```
class ConsistentHash {
  constructor(virtualNodes = 100) {
    this.virtualNodes = virtualNodes; // ఒక్కో server కి అనేక points (even distribution)
    this.ring = new Map(); // ringHash → server
    this.sortedHashes = [];
  }

  // 32-bit hash + avalanche finalizer (సారూప్య keys ని కూడా బాగా చెదరగొడుతుంది)
  #hash(str) {
    let h = 0x811c9dc5;
    for (let i = 0; i < str.length; i++) {
      h ^= str.charCodeAt(i);
      h = Math.imul(h, 0x01000193);
    }
    h ^= h >>> 16;
    h = Math.imul(h, 0x85ebca6b);
    h ^= h >>> 13;
    h = Math.imul(h, 0xc2b2ae35);
    h ^= h >>> 16;
    return h >>> 0;
  }

  addServer(server) {
    for (let v = 0; v < this.virtualNodes; v++)
      this.ring.set(this.#hash(`${server}#${v}`), server);
    this.sortedHashes = [...this.ring.keys()].sort((a, b) => a - b);
  }

  removeServer(server) {
    for (let v = 0; v < this.virtualNodes; v++)
      this.ring.delete(this.#hash(`${server}#${v}`));
    this.sortedHashes = [...this.ring.keys()].sort((a, b) => a - b);
  }

  // key ఏ server కి వెళ్తుంది – clockwise లో తర్వాతి ring point యొక్క server
  getServer(key) {
    const h = this.#hash(key);
    for (const ringHash of this.sortedHashes)
      if (h <= ringHash) return this.ring.get(ringHash);
    return this.ring.get(this.sortedHashes[0]); // wrap around (వృత్తం)
  }
```

```
}

const ch = new ConsistentHash();
["S1", "S2", "S3"].forEach((s) => ch.addServer(s));

const keys = Array.from({ length: 12 }, (_, i) => `user${i + 1}`);
const before = keys.map((k) => ch.getServer(k));

// keys 3 servers మీద పంచబడ్డాయి (perfectly even కాదు, కానీ చెదిరాయి)
const dist = {};
before.forEach((s) => (dist[s] = (dist[s] || 0) + 1));
console.log("Distribution:", JSON.stringify(dist)); // ఉదా: {"S2":5,"S3":6,"S1":1}

// S2 fail అయింది – కేవలం S2 keys మాత్రమే remap అవ్వాలి, S1/S3 keys stable
ch.removeServer("S2");
const after = keys.map((k) => ch.getServer(k));

const moved = keys.filter((k, i) => before[i] !== after[i]).length;
const s2Keys = before.filter((s) => s === "S2").length;
console.log(
  `S2 కి ${s2Keys} keys ఉన్నాయి; S2 తీసేస్తే సరిగ్గా ${moved} keys move అయ్యాయి`,
);
console.log(`S1/S3 keys అస్సలు కదల్లేదు: ${moved === s2Keys}`); // true
// `hash % N` అయితే దాదాపు అన్ని 12 keys move అయ్యేవి!
```

Virtual Nodes ఎందుకు

> ఒక్కో physical server ని ring మీద **అనేక points** (virtual nodes) గా పెడితే - load even గా పంచబడుతుంది. లేకపోతే కొన్ని servers కి ఎక్కువ, కొన్నిటికి తక్కువ traffic వస్తుంది.

Key Points

- `hash % N` సమస్య: node మారితే దాదాపు అన్ని keys remap → cache miss storm
- Consistent hashing: node add/remove → కేవలం `1/N` keys move
- Distributed cache (Memcached), Cassandra, DynamoDB, load balancers - అంతటా వాడతారు
- Virtual nodes = even load distribution

---

16. Replication Strategies + Quorum

వివరణ

Leaderless systems (Cassandra, Dynamo) లో data ని N nodes లో replicate చేసి, **Quorum** తో consistency సాధిస్తారు.

Quorum సూత్రం

> N = replicas సంఖ్య, W = write కి confirm కావలసిన nodes, R = read కి చదవాల్సిన nodes.
>
> **W + R > N** అయితే → strong consistency (read ఎప్పుడూ latest write ని చూస్తుంది, overlap ఉంటుంది).

QUORUM · R + W > N అయితే ఎందుకు తాజా DATA దొరుకుతుంది

N = 3 · W = 2 · R = 2 · R + W = 4 > 3 ✓

Replica 1	Replica 2	Replica 3
v2 (కొత్తది)	v2 (కొత్తది)	v1 (పాతది)

Write v2 → replicas 1, 2 ack ఇచ్చాయి (W = 2 సరిపోయింది). Replica 3 ఇంకా చేరుకోలేదు.

Read ఏవైనా 2 నుంచి (R = 2)
3 lo ఏ 2 తీసుకున్నా – కనీసం ఒకటి v2 ఉన్నదే ఉంటుంది

ఇదే pigeonhole principle
W nodes రాశాయి, R nodes చదువుతున్నాయి, R+W > N → overlap తప్పనిసరి

TUNING

W=1, R=1	W=3, R=1	W=1, R=3
వేగం, కానీ పాత data రావొచ్చు	Read-heavy కి · ఒక node పోతే write ఆగుతుంది	Write-heavy కి · reads ఖరీదు

ఇదే **tunable consistency** – ఒక్కో query కీ R, W ని మార్చొచ్చు. User profile చదవడానికి R=1 (వేగం), account balance కి R=2 (correctness). ఒకే database, రెండు రకాల హామీలు.

Code

```
// Quorum consistency check
function quorumCheck(N, W, R) {
  const strongConsistency = W + R > N;
  return {
    N,
    W,
    R,
    consistency: strongConsistency ? "Strong ✅" : "Eventual ⚠️",
    note: strongConsistency
      ? "Read, latest write ని guarantee గా చూస్తుంది (overlap)"
      : "Read stale data చూడవచ్చు",
  };
}

console.log(quorumCheck(3, 2, 2)); // W+R=4 > 3 → Strong (common config)
console.log(quorumCheck(3, 1, 1)); // W+R=2 < 3 → Eventual (fast, risky)
console.log(quorumCheck(3, 3, 1)); // W+R=4 > 3 → Strong (write-slow, read-fast)
```

Tuning W మరియు R

Config	లక్షణం
W=N, R=1	Fast reads, slow writes (read-heavy)
W=1, R=N	Fast writes, slow reads (write-heavy)
W=R=⌈(N+1)/2⌉	Balanced (N=3 → W=R=2, common)

Key Points

- W+R > N = strong consistency; W+R ≤ N = eventual (fast కానీ risky)
- W, R ని tune చేసి read/write speed vs consistency balance చేయవచ్చు
- Leaderless (Cassandra) systems దీని మీద నడుస్తాయి
- Conflict resolution: last-write-wins (timestamp) లేదా vector clocks

---

17. Message Queues & Kafka

వివరణ

Message Queue = producer ఒక పని ని queue లో పెడతాడు, consumer తర్వాత దాన్ని process చేస్తాడు - **asynchronous, decoupled**. Producer, consumer ఒకరి కోసం ఒకరు wait అవ్వరు.

Real-life Scenario

> **Restaurant లో order slips.** Waiter (producer) orders ని kitchen queue లో పెడతాడు. Cooks (consumers) వరుసగా తీసుకుంటారు. Waiter cook అయ్యేదాకా nిలబడడు - మరో order తీసుకుంటాడు. Rush వస్తే slips పేరుకుంటాయి (buffer), కానీ ఏదీ పోదు.

KAFKA · APPEND-ONLY LOG మరియు OFFSETS

Broker ఏ consumer ఏం చదివాడో track చేయదు

అది consumer బాధ్యత. ఈ ఒక్క నిర్ణయం వల్ల broker దాదాపు stateless అవుతుంది – అందుకే ఇది ఇంత scale అవుతుంది.
Offset ని వెనక్కి పెట్టి మళ్ళీ చదవొచ్చు – bug fix చేశాక 3 రోజుల data ని reprocess చేయడం ఉచితం

Partition = parallelism యొక్క unit మరియు ordering యొక్క unit – రెండూ ఒకటే. అందుకే partition key ఎంచుకోవడం అతి ముఖ్యమైన నిర్ణయం: `user_id` ఇస్తే ఒక user events క్రమంలో, వేరే users సమాంతరంగా.

ఎందుకు వాడతారు

ప్రయోజనం	వివరణ
Decoupling	Producer, consumer స్వతంత్రం
Async processing	Slow tasks (email, video encode) ని background కి
Load leveling	Traffic spike ని buffer చేసి, స్థిరంగా process
Reliability	Consumer crash అయినా message queue లో ఉంటుంది (retry)

Kafka (Event Streaming)

> Kafka = high-throughput distributed **log**. Messages **topics** లో, topics **partitions** గా (parallelism + scale). Consumers offset తో ఎక్కడ ఉన్నారో track చేస్తారు. Messages **persist** అవుతాయి (replay possible).

```
Producers → [ Topic: orders ]                  → Consumer Group
              ├ Partition 0: msg msg msg  ──────────> Consumer A
              ├ Partition 1: msg msg      ──────────> Consumer B
              └ Partition 2: msg msg msg  ──────────> Consumer C
              (partition = ordering + parallelism unit)
```

Queue vs Pub-Sub vs Kafka

	Message Queue	**Pub-Sub**	**Kafka (log)**
Delivery	ఒక consumer కి	అందరు subscribers కి	Consumer groups + replay
Message తర్వాత	Delete	Delete	Persist (retention)
ఉదా	RabbitMQ, SQS	Redis Pub/Sub	Kafka, Pulsar

Delivery Guarantees

Guarantee	**అర్థం**
At-most-once	ఒకసారి లేదా అస్సలు కాదు (loss possible)
At-least-once	కనీసం ఒకసారి (duplicates possible → idempotency అవసరం)
Exactly-once	సరిగ్గా ఒకసారి (కష్టం, ఖరీదు)

Key Points

- MQ = decoupling + async + reliability + load leveling (spikes ని smooth చేయడం)
- Kafka = high throughput, persistence, replay, ordering (per partition)
- At-least-once common → consumers **idempotent** గా ఉండాలి (Topic 18)

---

18. Idempotency

వివరణ

Idempotent operation = ఎన్నిసార్లు చేసినా ఫలితం ఒకటే. Distributed systems లో retries, duplicate messages సాధారణం - కాబట్టి operations idempotent గా ఉండాలి (లేకపోతే double charge, duplicate order).

Real-life Scenario

> **Lift button.** ఎన్నిసార్లు నొక్కినా lift ఒక్కసారే వస్తుంది - నొక్కిన సంఖ్య కాదు. అదే idempotency. కానీ "డబ్బు తీసుకో" 3 సార్లు retry అయితే 3 సార్లు తీస్తే ప్రమాదం!

Code

```
// Idempotency key తో duplicate requests ని safe గా handle చేయడం
class PaymentService {
  #processed = new Map(); // idempotencyKey → result (real లో Redis/DB)

  charge(idempotencyKey, userId, amount) {
    // ఇదే key ఇప్పటికే process అయితే — మళ్ళీ charge చేయకుండా పాత result ఇవ్వు
    if (this.#processed.has(idempotencyKey)) {
      console.log(`⏭️ Duplicate (${idempotencyKey}) — మళ్ళీ charge చేయలేదు`);
      return this.#processed.get(idempotencyKey);
    }
    const result = {
      status: "SUCCESS",
      userId,
      amount,
      txnId: `txn_${idempotencyKey}`,
    };
    this.#processed.set(idempotencyKey, result);
    console.log(`💳 Charged Rs.${amount} to ${userId}`);
    return result;
  }
}

const ps = new PaymentService();
// Client network timeout వల్ల same request 3 సార్లు పంపాడు — కానీ ఒక్కసారే charge
const key = "order_12345";
ps.charge(key, "user1", 500); // 💳 Charged Rs.500 to user1
ps.charge(key, "user1", 500); // ⏭️ Duplicate — మళ్ళీ charge చేయలేదు
ps.charge(key, "user1", 500); // ⏭️ Duplicate
```

ఎలా idempotent చేయాలి

- **Idempotency key** - client ఒక unique key పంపుతాడు; server అదే key ని రెండోసారి skip చేస్తుంది
- **Natural idempotency** - `SET x = 5` (ఎన్నిసార్లైనా ఒకటే); `x = x + 5` (కాదు!)
- **Dedup** - processed IDs ని store చేసి duplicates తీసేయడం

Key Points

- At-least-once delivery + retries = duplicates → idempotency తప్పనిసరి
- Payments, orders, inventory లో అతి ముఖ్యం (double-charge నివారణ)
- GET/PUT/DELETE naturally idempotent; POST కాదు (idempotency key అవసరం)

19. Distributed Transactions (2PC, Saga)

వివరణ

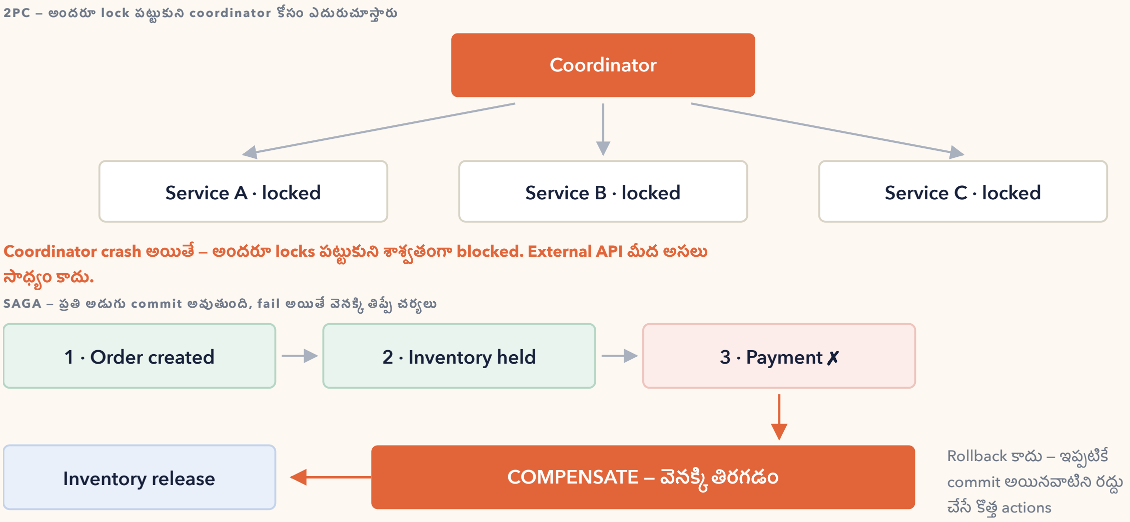
సమస్య: ఒక operation అనేక services/DBs ని touch చేస్తే (order → payment → inventory → shipping), అన్నీ కలిసి succeed లేదా అన్నీ fail అవ్వాలి. కానీ distributed లో single ACID transaction కుదరదు.

2PC (Two-Phase Commit)

```
Coordinator → అందరినీ అడుగుతుంది:
  Phase 1 (Prepare): "మీరు ready నా?" → అందరూ "yes/no"
  Phase 2 (Commit):  అందరూ yes అంటే "commit"; ఒక్కరు no అంటే "rollback"
```

- **సమస్య:** Coordinator fail అయితే అందరూ blocked (locks పట్టుకుని); slow, tight coupling

DISTRIBUTED TRANSACTIONS · 2PC VS SAGA



Compensation ≠ rollback. Payment already succeed అయి తర్వాత step fail అయితే, charge ని "undo" చేయలేం – REFUND అనే కొత్త transaction రాస్తాం. అందుకే **డబ్బు తీసే step ని saga lo ఎప్పుడూ చివరన** పెట్టాలి.

Saga Pattern (ఆధునిక పరిష్కారం)

> **Saga** = పెద్ద transaction ని చిన్న local transactions గా విభజించడం. ఒక్కో step succeed అవుతూ వెళ్తుంది; ఏదైనా fail అయితే, ముందు జరిగిన వాటిని **compensating actions** తో undo చేస్తారు.

```
Order Saga:
  1. Create Order      → (fail అయితే) Cancel Order
  2. Charge Payment    → (fail అయితే) Refund Payment
  3. Reserve Inventory → (fail అయితే) Release Inventory
  4. Ship

  Step 3 fail అయితే: Refund Payment + Cancel Order (వెనక్కి compensate)
```

Saga రకాలు

రకం	ఎలా
Choreography	ఒక్కో service event emit చేస్తుంది, తర్వాతిది react అవుతుంది (decentralized)
Orchestration	ఒక central orchestrator steps ని coordinate చేస్తుంది (centralized)

2PC vs Saga

	2PC	Saga
Consistency	Strong (atomic)	Eventual
Locks	పట్టుకుంటుంది (blocking)	లేదు (non-blocking)
Scale	తక్కువ	ఎక్కువ
Complexity	Coordinator	Compensating logic

Key Points

- Microservices లో distributed transaction కి **Saga** preferred (2PC కాదు - blocking)
- Compensating actions design చేయడం కష్టం కానీ కీలకం (refund, cancel...)
- Eventual consistency ని accept చేయాలి (కొంత సమయం inconsistent)

---

20. Consensus, Leader Election (Raft) + Distributed Locks

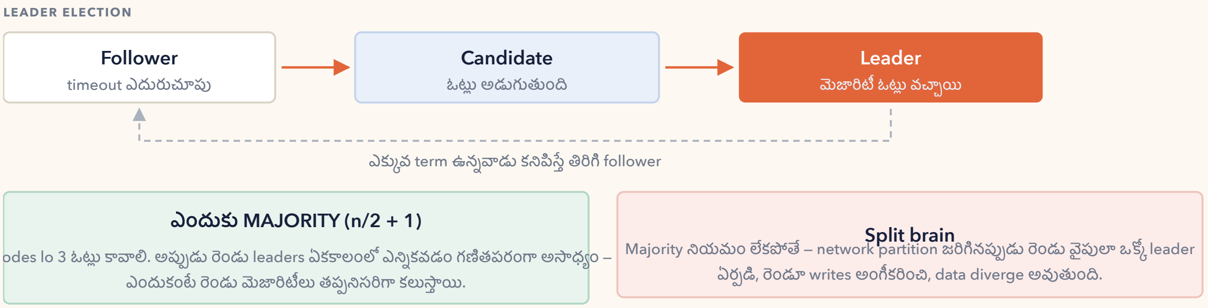
వివరణ

Consensus = అనేక nodes ఒకే విలువ మీద (ఉదా: "ఎవరు leader") ఏకాభిప్రాయానికి రావడం - కొన్ని nodes fail అయినా. **Raft, Paxos** - ఇందుకు algorithms.

Real-life Scenario

> **Team lead election.** Team members vote వేసి ఒక lead ని ఎన్నుకుంటారు (majority). Lead మాట్లాడకపోతే (crash), కొత్త election. అందరూ ఒకే lead ని అంగీకరించడం = consensus.

RAFT · LEADER ELECTION మరియు MAJORITY ఎందుకు



అందుకే cluster size ఎప్పుడూ **బేసి సంఖ్య** – 3, 5, 7. 4 nodes lo కూడా 3 ఓట్లే కావాలి, అంటే 4వ node వృథా.

Raft (సులభంగా)

```
– ప్రతి node: Follower / Candidate / Leader
– Leader crash → timeout → Follower "Candidate" అవుతుంది → votes అడుగుతుంది
– Majority votes వస్తే → Leader అవుతుంది
– Leader అన్ని writes ని coordinate చేసి followers కి replicate చేస్తుంది
– Majority (quorum) confirm అయితేనే commit
```

- **ఎందుకు majority?** Split-brain (రెండు leaders) నివారణకి. Majority = ఒకే leader guarantee.

Distributed Lock

> అనేక nodes ఒకే resource ని ఏకకాలంలో మార్చకుండా - ఒక్కరే lock పట్టుకోవాలి. **Redis (Redlock)** లేదా **Zookeeper** తో implement చేస్తారు.

- **జాగ్రత్త:** Lock holder crash అయితే? → **TTL** (auto-expire) పెట్టాలి, లేకపోతే deadlock
- **Fencing token** - stale lock holder వల్ల corruption నివారించడానికి

వీటిని ఎక్కడ వాడతారు

వాడకం	Tool
Leader election	Zookeeper, etcd, Raft
Config management	etcd, Consul
Distributed locks	Redis, Zookeeper
Service coordination	Zookeeper

Key Points

- Consensus = fault-tolerant agreement (Raft/Paxos); majority (quorum) కీలకం
- Leader election = single coordinator ని safely ఎన్నుకోవడం
- Distributed locks కి ఎప్పుడూ TTL + fencing token (crash safety)
- Zookeeper/etcd = ఈ coordination కి ready-made tools

Part 4 – Architecture & APIs

> Services ని ఎలా organize చేయాలి, ఒకదానితో ఒకటి ఎలా మాట్లాడాలి, failures ని ఎలా తట్టుకోవాలి - ఇవి architecture నిర్ణయాలు.

---

21. Monolith vs Microservices

వివరణ

- **Monolith** = మొత్తం application ఒకే codebase, ఒకే deployment unit.
- **Microservices** = application ని చిన్న, స్వతంత్ర services గా విభజించడం, ఒక్కోటి ఒక business capability.

Real-life Scenario

> **Monolith = ఒక పెద్ద డిపార్ట్‌మెంటల్ స్టోర్** (అన్నీ ఒకే భవనం). **Microservices = ఒక mall** లో వేర్వేరు shops (ఒక్కోటి స్వతంత్రం, ఒకటి మూసినా మిగతావి నడుస్తాయి).

పోలిక

అంశం	Monolith	Microservices
Deployment	ఒకే unit	ఒక్కో service విడిగా
Scaling	మొత్తం app ని scale	అవసరమైన service ని మాత్రమే
Tech stack	ఒకటే	ఒక్కో service కి వేరుగా
Failure	ఒక bug → మొత్తం down	ఒక service fail → మిగతావి OK (isolation)
Complexity	తక్కువ (మొదట్లో)	ఎక్కువ (network, distributed)
Team	చిన్న teams కి కష్టం (అందరూ ఒకే code)	Teams స్వతంత్రంగా పని చేయవచ్చు

ఎప్పుడు ఏది?

- **Monolith:** చిన్న team, startup, MVP, simple domain → **monolith తో మొదలుపెట్టు**
- **Microservices:** పెద్ద org, అనేక teams, వేర్వేరు scaling needs, matured product

> **SSE tip:** "Microservices always better" అనేది తప్పు. Premature microservices = distributed monolith (worst of both). **Monolith first**, అవసరమైనప్పుడు విడగొట్టు. Netflix, Amazon కూడా monolith నుండి మొదలుపెట్టాయి.

Key Points

- Microservices = independent scaling/deployment/tech, కానీ network + operational complexity
- Distributed systems సమస్యలన్నీ (latency, partial failure, consistency) microservices కి వర్తిస్తాయి
- Service boundaries = business capabilities (Domain-Driven Design)

---

22. API Gateway + Service Discovery

వివరణ

API Gateway = అన్ని client requests కి **single entry point**. Routing, auth, rate limiting, logging - వీటిని ఒక చోట handle చేస్తుంది.

Real-life Scenario

> **Hotel receptionist.** అతిథి నేరుగా kitchen/housekeeping కి వెళ్ళడు - receptionist (gateway) దగ్గరికి వెళ్తాడు. అతను సరైన department కి పంపుతాడు, ID check చేస్తాడు, అభ్యర్థనలు log చేస్తాడు.

API Gateway పనులు

పని	వివరణ
Routing	`/orders` → Order service, `/users` → User service
Authentication	Token verify (ఒక చోట, ప్రతి service లో కాదు)
Rate Limiting	అతిగా requests ని ఆపడం
Aggregation	అనేక services కలిపి ఒక response
SSL termination, logging, caching	Cross-cutting concerns

Service Discovery

> Microservices dynamic గా వస్తూ పోతూ ఉంటాయి (auto-scaling, crashes). ఒక service మరో దాన్ని ఎలా కనుక్కుంటుంది? **Service Registry** (Consul, etcd, Eureka) - services register అవుతాయి, ఇతరులు lookup చేస్తారు.

రకం	ఎలా
Client-side	Client registry ని అడిగి, నేరుగా service కి
Server-side	LB registry ని అడిగి route చేస్తుంది

Key Points

- API Gateway = single entry point + cross-cutting concerns (auth, rate limit, routing)
- కానీ Gateway single point of failure కాకూడదు → redundant + scaled
- Service Discovery = dynamic services ని కనుక్కోవడం (registry)

---

23. API Design (REST vs gRPC vs GraphQL)

వివరణ

Services ఎలా మాట్లాడాలి? API style ఎంపిక.

పోలిక

	REST	gRPC	GraphQL
Format	JSON over HTTP	Protobuf (binary)	JSON, query language
వేగం	Moderate	వేగం (binary, HTTP/2)	Moderate
Coupling	Loose	Tight (contract)	Flexible
Best for	Public APIs, CRUD	Internal microservices	Complex/nested data, mobile
Over/under-fetch	ఉంటుంది		Client కావలసినది మాత్రమే అడుగుతుంది

REST Best Practices

```
GET    /users/123        - fetch (idempotent, cacheable)
POST   /users            - create
PUT    /users/123        - full update (idempotent)
PATCH  /users/123        - partial update
DELETE /users/123        - delete (idempotent)

- Nouns వాడు (verbs కాదు): /users, /orders (not /getUsers)
- Status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error
- Versioning: /v1/users, /v2/users
- Pagination: /users?page=2&limit=20 (లేదా cursor-based)
```

ఎప్పుడు ఏది?

- **REST:** Public APIs, simple CRUD, wide compatibility
- **gRPC:** Internal service-to-service (fast, typed), low latency
- **GraphQL:** Mobile/frontend (bandwidth ముఖ్యం), nested/flexible data, అనేక sources aggregate

Key Points

- REST = default choice (simple, cacheable, universal)
- gRPC = internal microservices (binary, HTTP/2, streaming)
- GraphQL = over/under-fetching సమస్య పరిష్కారం (client shapes response)

---

24. Realtime (WebSockets, SSE, Long Polling)

వివరణ

Server నుండి client కి **realtime updates** (chat message, notification, live score) ఎలా పంపాలి? HTTP request-response ఇక్కడ సరిపోదు (client అడగాలి).

పద్ధతులు

పద్ధతి	ఎలా	వాడకం
Short Polling	Client ప్రతి X sec అడుగుతూ ఉంటుంది	Simple కానీ వృథా (చాలా empty responses)
Long Polling	Server data వచ్చేదాకా request ని hold	పాత browsers, moderate realtime
SSE (Server-Sent Events)	Server → client one-way stream	Notifications, live feed (unidirectional)
WebSocket	Full-duplex (రెండువైపులా) persistent connection	Chat, games, collaboration (bidirectional)

REALTIME · నాలుగు మార్గాలు

Short polling	ప్రతి 2s కి "ఏమైనా ఉందా?"	✗ భారీ వృథా traffic ✗ latency చెడ్డది
Long polling	Request ని server పట్టుకుని, data వచ్చినప్పుడు జవాబు	⚠ పని చేస్తుంది ప్రతి message తర్వాత reconnect
SSE	Server → client ఒక దిక్కు stream	✓ సులభం ✗ ఒక దిక్కు మాత్రమే
WebSocket	ఒకసారి handshake, తర్వాత full-duplex	✓ chat కి సరైనది ✗ servers stateful అవుతాయి

WebSocket యొక్క దాచిన ఖరీదు

Servers ఇప్పుడు stateful – "user 42 ఏ server తో connect అయ్యాడు?" అని తెలియాలి. అందుకే ఒక session registry (Redis) కావాలి, మరియు gateway crash అయితే clients reconnect చేయాలి.

Real-life Scenario

> - **Polling** = ప్రతి 5 నిమిషాలకి postman ని "ఉత్తరం వచ్చిందా?" అని అడగడం.
> - **Long polling** = postman "ఉత్తరం వచ్చాకే చెప్తా" అని wait చేయడం.
> - **WebSocket** = postman తో direct phone line (ఎప్పుడైనా రెండువైపులా మాట్లాడవచ్చు).

Key Points

- Bidirectional realtime (chat, games) → **WebSocket**
- Server → client one-way (notifications, feed) → **SSE** (simpler)
- WebSocket = stateful connection → scaling కష్టం (connection ఏ server లో ఉందో track చేయాలి)
- Millions of connections → dedicated connection servers + pub-sub backend

---

25. Event-Driven Architecture (CQRS, Event Sourcing)

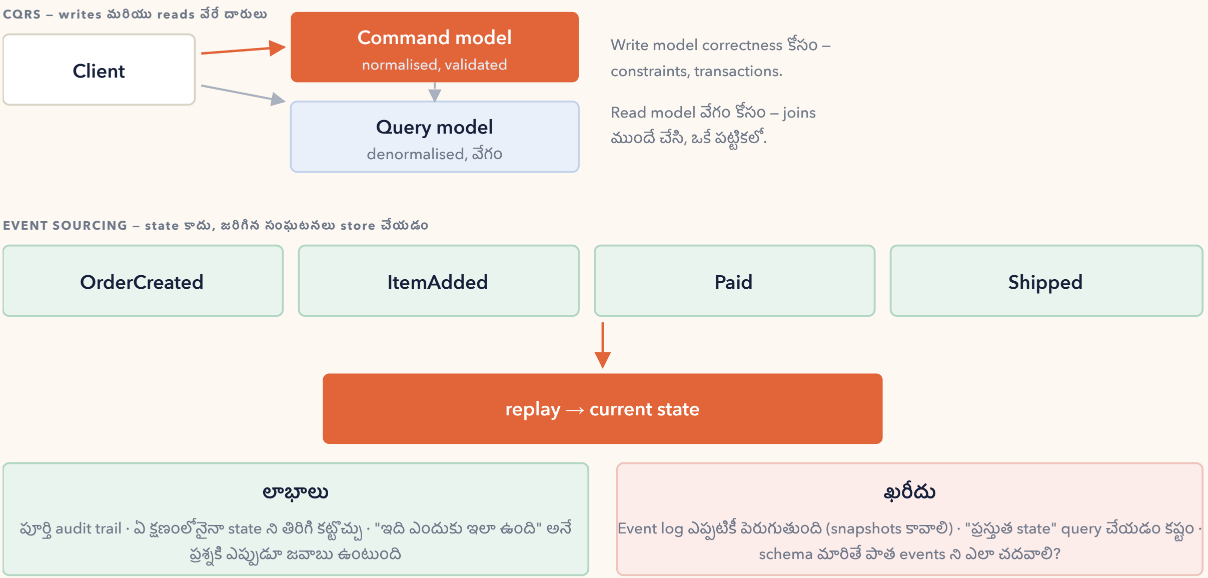
వివరణ

Event-Driven Architecture = services నేరుగా call చేసుకోకుండా, **events** ద్వారా react అవుతాయి (Kafka లాంటి bus ద్వారా). Loose coupling, scalability.

Real-life Scenario

> **Wedding.** Groom "పెళ్ళి అయింది" అని announce చేస్తాడు (event). Caterer, photographer, band - అందరూ react అవుతారు (వాళ్ళ పని మొదలుపెడతారు). Groom ప్రతి ఒక్కరినీ విడిగా పిలవడు.

CQRS + EVENT SOURCING



ఇవి రెండు వేర్వేరు ఆలోచనలు, తరచుగా కలిపి వాడతారు కానీ విడిగా కూడా వాడొచ్చు. CQRS లేకుండా event sourcing చేయొచ్చు, event sourcing లేకుండా CQRS చేయొచ్చు. రెండూ ఒకేసారి పెడితే సంక్లిష్టత రెట్టింపు – నిజంగా అవసరమా అని ఆలోచించండి.

CQRS (Command Query Responsibility Segregation)

> **Reads** మరియు **Writes** ని వేరు models గా విభజించడం. Write model (commands) normalize; Read model (queries) denormalize (fast reads). రెండూ వేర్వేరుగా scale అవుతాయి.

```
Commands (write) → Write DB → events → Read DB (denormalized) → Queries (read)
```

Event Sourcing

> State ని కాకుండా, **events sequence** ని store చేయడం. Current state = అన్ని events ని replay చేసి పొందడం.

> **ఉదా - Bank account:** balance=500 అని store చేయకుండా, `+1000, -300, -200` events store చేస్తాం. Balance = events sum. History అంతా ఉంటుంది (audit), ఏ point కైనా వెళ్ళవచ్చు.

ప్రయోజనాలు / సవాళ్ళు

ప్రయోజనం	సవాలు
Full audit trail (history)	Complexity ఎక్కువ
Time-travel (ఏ state కైనా)	Eventual consistency
Read/write independent scale	Event schema evolution కష్టం

Key Points

- Event-driven = loose coupling + async + scalability (Kafka backbone)
- CQRS = read/write models విడదీసి independent scaling (read-heavy systems)
- Event sourcing = state కి బదులు events store (audit, replay) - కానీ complex
- ఇవన్నీ eventual consistency తో వస్తాయి

---

26. Resilience (Circuit Breaker, Retry, Bulkhead, Rate Limiting)

వివరణ

Distributed systems లో failures **తప్పనిసరి**. ఒక service fail అయితే మొత్తం system పడిపోకుండా (cascading failure) కాపాడే patterns.

Circuit Breaker

> **Electrical fuse లాంటిది.** ఒక downstream service పదేపదే fail అవుతుంటే, circuit "open" అవుతుంది - ఆ service ని కొంతసేపు call చేయడం ఆపేస్తుంది (fail fast). కొంతసేపటికి "half-open" అయి test చేస్తుంది.

CIRCUIT BREAKER · మూడు STATES

Breaker లేకపోతే

Downstream నెమ్మదిస్తే ప్రతి call 30s timeout కోసం ఎదురుచూస్తుంది. Threads అన్నీ ఇరుక్కుంటాయి. మన service కూడా చస్తుంది – ఇదే cascading failure.

Breaker ఉంటే

Fast-fail – వెంటనే error. Threads ఖాళీగా ఉంటాయి, మిగతా features పని చేస్తాయి. Downstrea కి కోలుకునే సమయం దొరుకుతుంది.

జతగా వాడాల్సినవి: **timeout** (ఎంతసేపు ఎదురుచూడాలి), **retry with jitter** (jitter లేకపోతే అందరూ ఒకేసారి తిరిగి కొడతారు), మరియు **bulkhead** (ఒక downstream కి thread pool ని పరిమితం చేయడం).

Code

```
class CircuitBreaker {
  constructor(threshold = 3, cooldownMs = 1000) {
    this.threshold = threshold; // ఎన్ని failures తర్వాత open
    this.cooldownMs = cooldownMs;
    this.failures = 0;
    this.state = "CLOSED"; // CLOSED (normal) / OPEN (block) / HALF_OPEN (test)
    this.openedAt = 0;
  }

  async call(fn, nowMs) {
    // OPEN అయితే – cooldown అయ్యేదాకా fail fast (service ని రక్షించడం)
    if (this.state === "OPEN") {
      if (nowMs – this.openedAt < this.cooldownMs) {
        throw new Error("Circuit OPEN – fail fast (service ని call చేయలేదు)");
      }
      this.state = "HALF_OPEN"; // test చేద్దాం
    }
    try {
      const result = await fn();
      this.failures = 0;
      this.state = "CLOSED"; // success → normal
      return result;
    } catch (e) {
      this.failures++;
      if (this.failures >= this.threshold) {
        this.state = "OPEN";
        this.openedAt = nowMs;
      }
      throw e;
    }
  }
```

```
}

const breaker = new CircuitBreaker(3, 1000);
const failingService = async () => {
  throw new Error("service down");
};

// 3 failures తర్వాత circuit OPEN అవుతుంది
for (let t = 0; t < 5; t++) {
  try {
    await breaker.call(failingService, t * 100);
  } catch (e) {
    console.log(`t=${t * 100}ms: ${e.message} [state=${breaker.state}]`);
  }
}
// మొదటి 3 - "service down", తర్వాత "Circuit OPEN - fail fast"
```

మిగతా Resilience Patterns

Pattern	ఏం చేస్తుంది
Retry (with backoff)	Fail అయితే మళ్ళీ try (exponential backoff + jitter)
Timeout	ఎక్కువసేపు wait చేయకుండా fail
Bulkhead	Resources ని isolate (ఒక service అన్ని threads తినకుండా)
Rate Limiting	అతిగా requests ని ఆపడం (Token Bucket - LLD case study)
Fallback	Fail అయితే default/cached response

Key Points

- Circuit breaker = cascading failure నివారణ (fail fast, recover)
- Retry ఎప్పుడూ **backoff + jitter** తో (లేకపోతే thundering herd)
- Retry + idempotency కలిసి వాడాలి (duplicate నివారణ - Topic 18)
- Timeout లేని call = system hang కి ఆహ్వానం

---

27. Search (Inverted Index) + Bloom Filters

వివరణ

Full-text search (Google, Elasticsearch) కి **Inverted Index** వాడతారు. **Bloom Filter** = "ఇది ఉందా?" అని memory-efficient గా (కొంత error తో) చెప్పే probabilistic structure.

Inverted Index

> సాధారణ index: document → words. **Inverted** index: word → documents. "telugu" అనే word ఏ ఏ documents లో ఉందో నేరుగా map చేస్తుంది → search వేగం.

```
Documents:  doc1="telugu coding", doc2="system design", doc3="telugu design"

Inverted Index:
  "telugu" → [doc1, doc3]
  "coding" → [doc1]
  "system" → [doc2]
  "design" → [doc2, doc3]

Search "telugu" → వెంటనే [doc1, doc3]
```

INVERTED INDEX మరియు BLOOM FILTER

INVERTED INDEX – పదం → ఏ documents lo ఉంది

doc1: "system design guide"

doc2: "design patterns guide"

→

"design" → [doc1, doc2]
"system" → [doc1]
"patterns" → [doc2]

Search = పదాన్ని lookup చేసి, జాబితాలను intersect చేయడం. Documents అన్నీ scan చేయము – అదే వేగానికి కారణం.

BLOOM FILTER – "ఇది ఖచ్చితంగా లేదు" అని O(1) lo చెప్పడం

"లేదు" అంటే – ఖచ్చితంగా లేదు
False negative ఎప్పుడూ రాదు. అందుకే "disk కి వెళ్ళాలా?" అనే ప్రశ్నకి ఇది సరైన filter.

"ఉంది" అంటే – బహుశా ఉంది
False positive వస్తుంది. అప్పుడు నిజంగా చూడాలి. Bit array + k hash functions – memory చాలా తక్కువ.

వాడకం: LSM-tree read path (ఈ SSTable lo ఉందా?), web crawler URL dedupe, cache miss ముందు filter

Bloom Filter

> **సమస్య:** "ఈ username ఇప్పటికే ఉందా?" - ప్రతిసారి DB hit ఖరీదు. **Bloom filter** memory లో వేగంగా చెప్తుంది: "ఖచ్చితంగా లేదు" లేదా "బహుశా ఉంది". False positive ఉంటుంది, false negative ఉండదు.

Code

```
class BloomFilter {
  constructor(size = 100) {
    this.size = size;
    this.bits = new Array(size).fill(0);
  }

  #hashes(item) {
    // 3 వేర్వేరు hash functions (సులభ demo)
    let h1 = 0,
      h2 = 0,
      h3 = 7;
    for (let i = 0; i < item.length; i++) {
```

```
      const c = item.charCodeAt(i);
      h1 = (h1 * 31 + c) % this.size;
      h2 = (h2 * 37 + c) % this.size;
      h3 = (h3 * 41 + c) % this.size;
    }
    return [h1, h2, h3];
  }

  add(item) {
    this.#hashes(item).forEach((h) => (this.bits[h] = 1));
  }

  mightContain(item) {
    // అన్ని bits 1 అయితే "బహుశా ఉంది"; ఏ ఒక్కటి 0 అయినా "ఖచ్చితంగా లేదు"
    return this.#hashes(item).every((h) => this.bits[h] === 1);
  }
}

const bloom = new BloomFilter();
["yaswanth", "chaitanya", "priya"].forEach((u) => bloom.add(u));

console.log(bloom.mightContain("yaswanth")); // true  (ఉంది)
console.log(bloom.mightContain("chaitanya")); // true  (ఉంది)
console.log(bloom.mightContain("ravi")); // false (ఖచ్చితంగా లేదు - DB అడగక్కర్లేదు!)
console.log(bloom.mightContain("newuser")); // false (దాదాపు ఎప్పుడూ) - అరుదుగా false
positive
```

Bloom Filter ఎక్కడ

వాడకం	ఎలా సహాయం
Username availability	"లేదు" అయితే DB skip
Cache (does key exist?)	Cache miss ముందే తెలుసుకోవడం
Web crawler	URL ఇప్పటికే crawl అయిందా?
Databases (Cassandra)	SSTable లో key ఉందా?

Key Points

- Inverted index = word → documents (full-text search backbone; Elasticsearch)
- Bloom filter = "ఖచ్చితంగా లేదు" లేదా "బహుశా ఉంది" (false positive ok, false negative కాదు)
- Bloom filter memory ఆదా చేస్తుంది (అనవసర DB/disk lookups తగ్గిస్తుంది)

Part 5 — Reliability & Operations

> Design అయిపోతే సరిపోదు - system **production లో** ఎలా నడుస్తుంది, fail అయితే ఏమవుతుంది, ఎలా monitor చేస్తాం. SSE level లో ఇవి తప్పనిసరి.

---

28. Availability (9s), SLA / SLO / SLI

వివరణ

Availability = system up & working ఉన్న సమయ శాతం. "Nines" లో కొలుస్తారు.

The Nines

Availability	Downtime/year	పేరు
99%	~3.65 రోజులు	"two nines"
99.9%	~8.75 గంటలు	"three nines"
99.99%	~52 నిమిషాలు	"four nines"
99.999%	~5 నిమిషాలు	"five nines"

> ఒక్కో "nine" జోడించడం **ఖరీదు గణనీయంగా పెరుగుతుంది** (redundancy, multi-region). 99.99% చాలా systems కి సరిపోతుంది.

SLA vs SLO vs SLI

పదం	అర్థం	ఉదా
SLI (Indicator)	ఏం కొలుస్తాం (metric)	p99 latency, error rate
SLO (Objective)	మన లక్ష్యం (internal)	"p99 < 200ms, 99.9% uptime"
SLA (Agreement)	customer తో ఒప్పందం (legal, penalty)	"99.9% లేకపోతే refund"

> **సంబంధం:** SLI కొలుస్తాం → SLO లక్ష్యంగా పెట్టుకుంటాం → SLA customer కి promise చేస్తాం. SLO ఎప్పుడూ SLA కంటే కఠినంగా ఉంచుతారు (buffer).

Availability ఎలా పెంచాలి

- **Redundancy** - single point of failure తీసేయడం (అనేక servers, LBs, DB replicas)

- **Failover** - fail అయితే automatic గా backup కి switch
- **Multi-region** - ఒక region down అయినా మరో region
- **Health checks + auto-healing** - fail అయినవి restart/replace

Key Points

- Availability = redundancy (SPOF తీసేయడం) ద్వారా వస్తుంది
- SLI (కొలత) → SLO (లక్ష్యం) → SLA (ఒప్పందం)
- 100% అసాధ్యం - ఎంత "nines" అవసరమో business నిర్ణయిస్తుంది (ఖరీదు trade-off)

---

29. Observability (Metrics, Logs, Traces)

వివరణ

Observability = production లో system లోపల ఏం జరుగుతోందో బయటి signals నుండి అర్థం చేసుకోగలగడం. "3 pillars": Metrics, Logs, Traces.

3 Pillars

Pillar	ఏం చెప్తుంది	ఉదా
Metrics	సంఖ్యలు (aggregated) - "ఎంత?"	QPS, latency p99, error rate, CPU
Logs	Events (discrete) - "ఏం జరిగింది?"	"User 123 login failed at 10:05"
Traces	ఒక request యొక్క journey - "ఎక్కడ నెమ్మది?"	Request → gateway → svc A → svc B → DB

Real-life Scenario

> **డాక్టర్ diagnosis:** Metrics = జ్వరం/BP సంఖ్యలు (ఏదో తేడా ఉంది). Logs = రోగి చెప్పే వివరాలు (ఏం జరిగింది). Traces = X-ray/scan (సరిగ్గా ఎక్కడ సమస్య).

Distributed Tracing

> Microservices లో ఒక request 10 services గుండా వెళ్తుంది. **Trace ID** ప్రతి service కి pass అవుతుంది - మొత్తం journey ని కలిపి చూడవచ్చు (ఏ service నెమ్మది చేసిందో). Jaeger, Zipkin.

Alerting

- Metrics మీద thresholds → alerts ("error rate > 5% → page on-call")
- **Alert fatigue** జాగ్రత్త - అతిగా alerts = ముఖ్యమైనవి miss

Key Points

- Metrics (ఎంత) + Logs (ఏం) + Traces (ఎక్కడ) = complete picture

- Distributed tracing = microservices లో bottleneck కనుక్కోవడానికి కీలకం
- Monitor చేయలేనిది operate చేయలేవు - observability design లో భాగం
- Prometheus (metrics), ELK/Loki (logs), Jaeger (traces)

---

30. Deployment (Blue-Green, Canary, Rolling)

వివరణ

కొత్త version ని production కి deploy చేసేటప్పుడు **downtime లేకుండా, risk తక్కువగా** ఎలా చేయాలి.

Deployment Strategies

Strategy	ఎలా	Risk
Rolling	ఒక్కో server ని వరుసగా update	Moderate (mixed versions కొంతసేపు)
Blue-Green	2 identical envs (blue=live, green=new); switch traffic	తక్కువ (instant rollback)
Canary	కొత్త version ని కొద్ది % users కి మాత్రమే → monitor → పెంచడం	అతి తక్కువ (early detection)

Real-life Scenario

> **Canary = బొగ్గు గనిలో పక్షి.** ముందు కొద్దిమంది users కి కొత్త version ఇచ్చి, సమస్య వస్తే వాళ్ళకే (కొద్దిమందికే) - అందరికీ కాదు. Safe అయితే అందరికీ.

Blue-Green వివరంగా

```
Blue (v1, live) ← 100% traffic
Green (v2, new) ← 0% (test చేస్తున్నాం)

Deploy: Green ని test → traffic ని Green కి switch (instant)
Problem? → వెంటనే Blue కి తిరిగి switch (rollback)
```

Key Points

- Rolling = resource-efficient కానీ mixed versions; Blue-Green = instant rollback కానీ 2x resources
- Canary = safest (gradual, early problem detection) - పెద్ద systems కి preferred
- Feature flags = code deploy ని feature release నుండి విడదీయడం
- ఎప్పుడూ rollback plan ఉండాలి

---

31. Failover & Disaster Recovery

వివరణ

Failover = ఒక component fail అయితే backup కి switch. **Disaster Recovery (DR)** = మొత్తం datacenter/region పోతే recover చేయడం.

Failover రకాలు

రకం	ఎలా	Cost
Active-Passive	Backup standby లో (idle); fail అయితే activate	తక్కువ (backup idle)
Active-Active	రెండూ live, traffic పంచుతాయి; ఒకటి fail → మరోటి	ఎక్కువ కానీ 0 downtime

DR Metrics

Metric	అర్థం
RPO (Recovery Point Objective)	ఎంత data loss ఓకే? (backup frequency)
RTO (Recovery Time Objective)	ఎంత సమయంలో recover అవ్వాలి?

> **ఉదా:** RPO=1hr అంటే గరిష్ఠంగా 1 గంట data పోవచ్చు (గంటకోసారి backup). RTO=5min అంటే 5 నిమిషాల్లో తిరిగి రావాలి.

Backups + Multi-Region

- **Backups** - regular, tested (test చేయని backup = backup కాదు!)
- **Replication across regions** - ఒక region down అయినా మరో region
- **Multi-region active-active** - highest availability (కానీ data consistency కష్టం)

Key Points

- Failover = component-level; DR = region/datacenter-level recovery
- RPO (data loss tolerance) + RTO (recovery time) = DR ని define చేస్తాయి
- Backups ని regularly **test** చేయాలి (restore పని చేస్తుందా?)
- Active-active = 0 downtime కానీ ఖరీదు + consistency challenges

Part 6 – Case Studies (System Design)

> ఇప్పుడు నేర్చుకున్నవన్నీ కలిపి నిజమైన systems design చేద్దాం. ప్రతి case study: **Requirements** → **Estimation** → **API** → **Data Model** → **Architecture** → **Deep Dive** → **Bottlenecks**. ఇదే interview flow.

ఈ PART గురించి ఒక ముఖ్యమైన మాట

కింది case studies ఇక్కడ **సంక్షిప్తంగా** ఉన్నాయి – నేర్చుకున్న building blocks ని ఒక నిజమైన systemలో ఎలా కలుపుతారో చూపించడానికి.

ఇవే problems `HLD_Design_Problems_Telugu.pdf` lo చాలా లోతుగా ఉన్నాయి – ఒక్కో దానికి 6–11 పేజీలు: clarifying questions, పూర్తి capacity estimation, architecture diagrams, deep dive, failure analysis, మరియు **interview lo నోటితో చెప్పాల్సిన English script**.

ఎలా వాడాలి: ఈ document = *పరికరాల పెట్టె* (caching అంటే ఏమిటి, quorum అంటే ఏమిటి). ఆ book = *ఆ పరికరాలతో 18 నిజమైన problems ని ఎలా పరిష్కరించాలి*. ముందు ఇది, తర్వాత అది.

---

32. URL Shortener (TinyURL / bit.ly)

1. Requirements

- **Functional:** పొడవైన URL → చిన్న URL; చిన్న URL → redirect; (optional) custom alias, expiry, analytics
- **Non-functional:** High availability, low latency (redirect fast), read-heavy (100:1)

2. Estimation

- 100M new URLs/day → ~1,160 writes/sec
- Read:Write = 100:1 → ~116,000 reads/sec
- 5 years storage: 100M × 365 × 5 ≈ 182B URLs → 62^7 (~3.5 trillion) short codes సరిపోతాయి

3. API

```
POST /shorten { longUrl, customAlias?, expiry? } → { shortUrl }
GET  /{shortCode}  → 301 Redirect to longUrl
```

4. Data Model (NoSQL - simple key-value, massive scale)

```
shortCode (PK) | longUrl | createdAt | expiry | userId
```

```
"aB3xK9z"      | "https://..." | ... | ... | ...
```

5. Short Code ఎలా generate చేయాలి?

పద్ధతి	వివరణ
Counter + Base62	Global counter → base62 encode (short, unique, no collision)
Hash (MD5) + take 7 chars	Collision possible → check & retry
Random + check	Random 7 chars, DB లో ఉందా check

> **మంచిది:** Distributed counter (Redis/Zookeeper/range-allocation) → Base62 encode. Collision లేదు, guaranteed short.

Code - Base62 Encoding

```
const CHARS = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"; // 62

// Counter (unique number) → short code (base62)
function encodeBase62(num) {
  if (num === 0) return CHARS[0];
  let code = "";
  while (num > 0) {
    code = CHARS[num % 62] + code;
    num = Math.floor(num / 62);
  }
  return code;
}

function decodeBase62(code) {
  let num = 0;
  for (const ch of code) num = num * 62 + CHARS.indexOf(ch);
  return num;
}

// ఒక్కో కొత్త URL కి counter++ → unique short code (collision లేదు!)
console.log(encodeBase62(1)); // "1"
console.log(encodeBase62(125)); // "21"
console.log(encodeBase62(1000000000)); // "15FTGg" (10 అంకెల ID → 6 అక్షరాలు!)
console.log(decodeBase62("15FTGg")); // 1000000000 (తిరిగి - case-sensitive!)

// 7 base62 chars = 62^7 ≈ 3.5 trillion URLs (చాలు!)
console.log(`62^7 = ${(62 ** 7).toLocaleString()} URLs possible`);
```

6. Architecture

```
Client → LB → App Servers → Cache (Redis: shortCode→longUrl) → DB (NoSQL)
                               ↑ 116K reads/sec మెజారిటీ cache నుండి
Write: counter service (unique ID) → Base62 → store
```

7. Deep Dive / Bottlenecks

- **Read-heavy** → aggressive caching (hot URLs). Cache hit ~95%+
- **Redirect** = 301 (permanent, browser caches) vs 302 (temporary, analytics కి)
- **Counter bottleneck** → range-based allocation (ఒక్కో server కి 1000 IDs block ఇవ్వడం)
- **Analytics** → async (Kafka event → separate analytics DB), redirect ని slow చేయకుండా

> **Trade-off:** 301 (fast, browser cache) vs 302 (analytics track చేయవచ్చు కానీ ప్రతిసారి server hit).

---

33. Distributed Rate Limiter

1. Requirements

- User/IP కి నిర్ణీత సమయంలో X requests మాత్రమే
- Distributed (అనేక servers మధ్య shared limit)
- Low latency (ప్రతి request ముందు check)

2. Algorithm ఎంపిక

Algorithm	లక్షణం
Token Bucket	Bursts allow (tokens పోగుపడితే) - popular
Leaky Bucket	స్థిర rate (smooth)
Fixed Window	Simple కానీ boundary spike సమస్య
Sliding Window	Accurate, boundary సమస్య లేదు

(Token Bucket implementation కోసం `LLD_Telugu.md` §36 Rate Limiter చూడు.)

3. Distributed ఎలా?

> **సమస్య:** 10 app servers, ఒక్కోటి local counter → user 10× limit దాటవచ్చు! **పరిష్కారం:** **Redis** (centralized) లో counter. అన్ని servers Redis ని atomic గా increment చేస్తాయి (`INCR`). ఒకే source of truth.

```
Request → App Server → Redis (atomic INCR user:123)
                        count > limit? → 429 Too Many Requests
                        else → allow
```

4. Architecture

```
Client → API Gateway (rate limit check here) → Redis (counters)
                     429 if exceeded ↑         (TTL తో window reset)
```

5. Deep Dive / Bottlenecks

- **Redis single point** → Redis cluster + replication
- **Latency** → Redis call ప్రతి request కి (fast కానీ overhead) - local cache + sync hybrid
- **Race condition** → Redis atomic ops (INCR, Lua scripts) వాడాలి
- **Fail-open vs fail-closed** → Redis down అయితే? allow అన్నీ (fail-open, availability) vs block (fail-closed, safety)

> **Trade-off:** Accuracy (centralized Redis, slower) vs performance (local approximate limits, faster).

---

34. News Feed (Twitter / Instagram)

1. Requirements

- User తను follow చేసేవారి posts ని timeline లో చూడాలి (recent first)
- Post create, follow/unfollow
- Read-heavy, low latency feed

2. Estimation

- 100M DAU, ఒక్కో feed request 20 posts → భారీ read QPS
- Celebrity problem: ఒక్కో celebrity కి 100M followers

3. Core సమస్య: Fan-out (feed ఎలా build చేయాలి?)

Approach	ఎలా	Trade-off
Fan-out on Write (Push)	Post చేసినప్పుడే అందరి followers feed లో పెట్టడం	Read fast, కానీ celebrity కి write భారీ
Fan-out on Read (Pull)	Feed అడిగినప్పుడు following posts collect	Write light, కానీ read slow
Hybrid (best)	Normal users → push; celebrities → pull	రెండింటి బలం

4. Hybrid Architecture (SSE answer)

```
Normal user posts → Fan-out service → అందరి followers feed cache లో push (Redis list)
Celebrity posts   → push చేయరు; user feed load అయినప్పుడు celebrity posts ని merge
(pull)
```

```
Feed Read: Precomputed feed (cache) + celebrity posts (pull) → merge → return
```

5. Data Model

```
Posts (NoSQL):     postId | userId | content | createdAt
Follows (NoSQL):   userId | followsUserId
Feed Cache (Redis): userId → [postId, postId, ...] (precomputed, recent N)
```

6. Deep Dive / Bottlenecks

- **Celebrity fan-out** → hybrid (celebrities ని push చేయకుండా read time లో merge)
- **Feed storage** → Redis లో recent ~1000 posts మాత్రమే (పాతవి DB నుండి)
- **Ranking** → time-order simple; ML ranking (engagement) complex
- **Read QPS** → feed precomputed + cached (fan-out on write వల్ల read fast)

> **SSE insight:** ఒక్క approach సరిపోదు. **Hybrid** - "80% users కి push (fast reads), celebrities కి pull (write explosion నివారణ)". ఇది trade-off reasoning చూపిస్తుంది.

---

35. Chat System (WhatsApp / Messenger)

1. Requirements

- 1-1 messaging, group chat, online/offline status, delivery receipts (sent/delivered/read)
- Realtime, message ordering, history

2. Core: Realtime Connection

> **WebSocket** persistent connections (Topic 24). ప్రతి online user ఒక connection server కి connected. Message routing: sender's server → recipient's connection server → recipient.

3. Architecture

```
User A ──WebSocket──> Connection Server 1 ┐
                                          ├→ Message Service → Kafka → DB (persist)
User B ──WebSocket──> Connection Server 2 ┘         │
                                                    ↓
      "User B ఏ server లో?" → Session Registry (Redis: userId → serverId)
      → route message to Connection Server 2 → deliver to User B
```

4. Data Model

```
Messages (NoSQL, sharded by chatId):
  chatId | messageId | senderId | content | timestamp | status
Sessions (Redis): userId → connectionServerId (ఎవరు ఏ server లో)
```

5. Deep Dive

- **User offline** → message DB లో store; online అయినప్పుడు deliver (+ push notification)
- **Ordering** → per-chat sequence number / timestamp (shard by chatId → ordering ఒక shard లో)
- **Delivery receipts** → sent (server got), delivered (device got), read (opened) - ack events
- **Group chat** → message ని అందరు members కి fan-out (చిన్న groups push)
- **Scale connections** → millions WebSockets → అనేక connection servers + session registry

> **Trade-off:** Message ordering (strong, per-chat) vs global ordering (అనవసరం, ఖరీదు). Shard by chatId → ordering సులభం.

---

36. Video Streaming (YouTube / Netflix)

1. Requirements

- Video upload, encode, stream (adaptive quality); massive scale, low buffering
- Read-heavy (views >> uploads)

2. Core: Upload → Encode → Serve

```
Upload → Blob Storage (S3) → Transcoding (అనేక resolutions: 240p...4K)
       → Chunking (HLS/DASH: video ని చిన్న segments గా) → CDN
Play → CDN edge (user దగ్గర) → adaptive bitrate (network బట్టి quality)
```

3. Architecture

```
Upload Service → Queue (Kafka) → Transcoding Workers (parallel) → Blob Storage
                                                                       ↓
Client (player) → CDN (video chunks) ← origin (Blob Storage)
Metadata (title, views) → DB + Cache
```

4. Key Concepts

Concept	వివరణ
Transcoding	ఒక video → అనేక resolutions/formats (devices కి)

Concept	వివరణ
Adaptive Bitrate (ABR)	Network slow → quality తగ్గించడం (buffering నివారణ)
Chunking (HLS/DASH)	Video ని 2-10s segments గా (progressive load)
CDN	Video chunks ని user దగ్గర serve (Topic 8) - తప్పనిసరి

5. Deep Dive / Bottlenecks

- **Storage** → పెద్దది (అనేక resolutions × millions videos) → blob storage + tiered (hot/cold)
- **Transcoding** → CPU-heavy, async (queue + parallel workers)
- **Bandwidth** → CDN (origin ని కాపాడుతుంది, latency తగ్గిస్తుంది)
- **Popular videos** → aggressively CDN-cached; long-tail → origin

> **SSE insight:** Video = 90% CDN + storage problem. Encoding pipeline (async, parallel) + adaptive streaming + global CDN = core. View counts eventual consistency ఓకే.

---

37. Notification System

1. Requirements

- Push (mobile), Email, SMS - అనేక channels
- High throughput, reliability (miss అవ్వకూడదు), user preferences

2. Architecture (Queue-based)

```
Services → Notification API → Kafka (queue) → Workers → Channel adapters
(Push/Email/SMS)
                                                        ↓ (3rd party: FCM, SES,
Twilio)
           User Preferences DB (ఏ channel, opt-out?)
           Template Service (message templates)
```

3. Key Design Points

అంశం	పరిష్కారం
Reliability	Queue (Kafka) - worker crash అయినా message ఉంటుంది; retry
Rate limiting	3rd party (FCM/Twilio) limits → throttle
Idempotency	Duplicate notification నివారణ (Topic 18)
Preferences	User opt-out, channel choice, quiet hours

అంశం	పరిష్కారం
Priority	OTP (high) vs promotional (low) → separate queues
Fan-out	ఒక event → millions users (batch + parallel workers)

4. Deep Dive

- **Async decoupling** → services notification పంపి వెంటనే return (Kafka లో); workers process
- **Retry + DLQ** → fail అయితే retry; పదేపదే fail → Dead Letter Queue (manual review)
- **Template + personalization** → message templates + user data merge
- **Analytics** → sent/delivered/opened tracking (async)

> **Trade-off:** At-least-once (duplicates possible, idempotency అవసరం) vs exactly-once (ఖరీదు). చాలావరకు at-least-once + idempotency.

---

38. Uber / Ride-Sharing (Geo)

1. Requirements

- Rider దగ్గరి drivers ని కనుక్కోవడం; matching; live location tracking; ETA
- Realtime, geo-spatial queries, low latency

2. Core సమస్య: "దగ్గరి drivers ఎలా కనుక్కోవాలి?"

> Millions drivers, ప్రతి 4 sec location update. "ఈ rider కి 2km లో ఏ drivers?" - naive గా అందరి distance లెక్కించడం అసాధ్యం. **Geo-spatial indexing** అవసరం.

Technique	వివరణ
Geohash	Earth ని grid cells గా, ఒక్కో cell కి string code. దగ్గరి = same/adjacent cell
Quadtree	Recursive గా space ని 4 quadrants గా (dense areas deeper)
S2 (Google)	Sphere ని cells గా (Uber, Google Maps వాడతాయి)

3. Architecture

```
Driver app → location updates (every 4s) → Location Service → Redis (Geo: geohash →
drivers)
Rider request → Matching Service → nearby drivers (geohash query) → match → notify
driver
Both → WebSocket (live tracking)
```

4. Data Model

```
Driver Location (Redis GEO): driverId → (lat, lng), geohash
Trips (DB): tripId | riderId | driverId | status | route
```

5. Deep Dive / Bottlenecks

- **Location updates** → భారీ write QPS (millions × every 4s) → in-memory (Redis), not disk DB
- **Geo query** → geohash/quadtree తో దగ్గరి cell drivers మాత్రమే (అందరూ కాదు)
- **Matching** → nearby drivers + ETA + supply/demand (surge pricing)
- **Hotspots** → busy areas (airport) → cell దగ్గర ఎక్కువ drivers → dynamic cell size (quadtree)
- **Consistency** → driver location eventual ఓకే (కొన్ని seconds పాతది ఫర్వాలేదు)

> **SSE insight:** Geo-spatial indexing (geohash/quadtree) = core. Location = high write, in-memory, eventual consistency. Matching = geo query + business logic (ETA, surge).

Part 7 – SSE Deep Dives

> SSE interview లో ఒక్కో topic మీద interviewer **లోతుగా** తవ్వుతాడు - "అది లోపల ఎలా పనిచేస్తుంది?", "ఇది fail అయితే?". ఈ part అదే లోతు - storage internals, isolation, stream processing, multi-region, security, cost. ఇవి SDE2 ని SSE నుండి వేరు చేస్తాయి.

---

39. Storage Internals (LSM-tree vs B-tree)

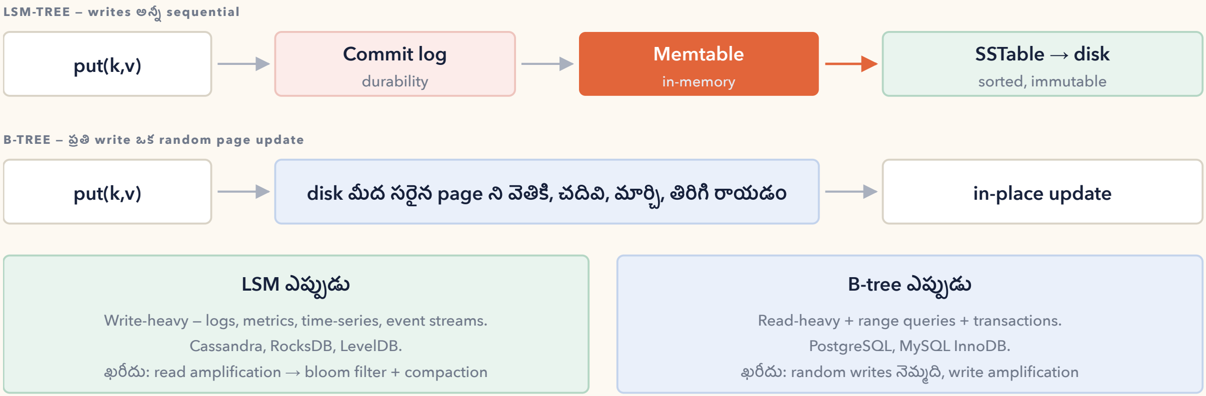
వివరణ

"NoSQL fast writes, SQL fast reads" అని ఎందుకు? లోపల **storage engine** వేరు. B-tree (reads కి) vs LSM-tree (writes కి) - ఈ తేడా తెలిస్తే DB ఎంపిక justify చేయవచ్చు.

B-Tree (SQL - MySQL, Postgres)

> Data ని sorted tree లో **in-place** update చేస్తుంది. Read వేగం (O(log n), direct seek). కానీ write = random disk seek (నెమ్మది).

STORAGE INTERNALS · LSM-TREE VS B-TREE



ఈ ఒక్క ఎంపిక database యొక్క వ్యక్తిత్వాన్ని నిర్ణయిస్తుంది. "Cassandra writes ఎందుకు ఇంత వేగం?" – ఎందుకంటే అది ఒక log కి append చేస్తోంది, disk మీద page ని వెతకడం లేదు.

LSM-Tree (NoSQL - Cassandra, RocksDB, LevelDB)

> Writes ని ముందు in-memory **memtable** లో (sequential, fast). నిండాక disk కి immutable **SSTable** గా flush. Read = memtable + SSTables వెతకడం (bloom filter సహాయం). Writes చాలా fast, reads కొంత నెమ్మది.

Code - LSM ఆలోచన (simplified)

```
class LSMStore {
  #memtable = new Map(); // in-memory (fast, sequential writes)
  #sstables = []; // disk segments (immutable, sorted) — newest first
  #threshold;
  constructor(threshold = 3) {
    this.#threshold = threshold;
  }

  put(key, value) {
    this.#memtable.set(key, value); // write ఎప్పుడూ memory కి (fast)
    if (this.#memtable.size >= this.#threshold) this.#flush();
  }
  #flush() {
    // memtable → sorted immutable SSTable, disk కి (newest ముందు)
    this.#sstables.unshift(new Map([...this.#memtable].sort()));
    console.log(
      `📥 Flushed ${this.#memtable.size} keys to SSTable
#${this.#sstables.length}`,
    );
    this.#memtable = new Map();
  }
  get(key) {
    if (this.#memtable.has(key)) return this.#memtable.get(key); // memory ముందు
    for (const sst of this.#sstables) if (sst.has(key)) return sst.get(key); //
newest → oldest
    return null;
  }
}

const db = new LSMStore(3);
db.put("a", 1);
db.put("b", 2);
db.put("c", 3); // 3 keys → flush
db.put("a", 10); // కొత్త value memtable లో (పాత "a"=1 పాత SSTable లో ఉన్నా)
console.log("get a:", db.get("a")); // 10 (memtable newest wins)
console.log("get b:", db.get("b")); // 2 (SSTable నుండి)
```

ముఖ్య Concepts

Concept	వివరణ
WAL (Write-Ahead Log)	Memtable కి రాసేముందు append-only log కి (crash recovery)
Compaction	పాత SSTables ని merge చేసి duplicates/deletes తీసేయడం (background)
Write Amplification	ఒక write, compaction వల్ల అనేకసార్లు disk కి రాయబడటం
Tombstone	Delete = మార్క్ (immediate కాదు); compaction లో నిజంగా తీసేస్తారు

B-Tree vs LSM-Tree

	B-Tree (SQL)	LSM-Tree (NoSQL)
Writes	నెమ్మది (random, in-place)	వేగం (sequential, append)
Reads	వేగం (direct)	కొంత నెమ్మది (అనేక SSTables + bloom)
Write amplification	తక్కువ	ఎక్కువ (compaction)
ఉదా	MySQL InnoDB, Postgres	Cassandra, RocksDB, HBase

> **SSE insight:** "Cassandra write-heavy కి మంచిది" అంటే సరిపోదు - "**LSM-tree** వల్ల writes sequential (append), అందుకే write-heavy కి. కానీ reads కి అనేక SSTables వెతకాలి - bloom filters సహాయం" అని ఎందుకో చెప్పు.

---

40. Isolation Levels & Consistency Depth

వివరణ

Topic 14 లో consistency models చూశాం. SSE level లో **database isolation levels** (ACID లో "I") మరియు **serializability vs linearizability** తేడా అడుగుతారు.

Isolation Levels (weak → strong)

Level	ఏం అనుమతిస్తుంది (anomaly)
Read Uncommitted	Dirty reads (commit కాని data చదవడం)
Read Committed	Committed data మాత్రమే; కానీ non-repeatable reads
Repeatable Read	ఒకే row మళ్ళీ చదివితే same; కానీ phantom reads
Serializable	పూర్తి isolation (transactions serial గా run అయినట్టు)

Concurrency Anomalies

Anomaly	అర్థం
Dirty Read	మరో transaction commit కాని change చదవడం
Non-repeatable Read	ఒకే query రెండుసార్లు వేర్వేరు results
Phantom Read	Range query కి కొత్త rows మధ్యలో వచ్చాయి
Write Skew	ఇద్దరూ చదివి, ఇద్దరూ రాసి, invariant break

Serializability vs Linearizability (SSE favourite)

	Serializability	Linearizability
దేని గురించి	**Transactions** (అనేక ops)	**ఒక్క object** (single op)
Guarantee	Transactions ఏదో ఒక serial order లో జరిగినట్లు	Ops real-time order లో, ఒక్కసారే జరిగినట్లు
Layer	Database isolation	Distributed register/consensus
కలిపి	**Strict Serializability** = రెండూ (Spanner)	

> **సులభంగా:** Serializability = "results ఏదో ఒక వరుసకి సరిపోతాయి" (real-time order కాదు). Linearizability = "ప్రతి op అది జరిగిన క్షణంలోనే అందరికీ కనిపిస్తుంది" (real-time). Google Spanner = strict serializability (రెండూ) - TrueTime వాడి.

Snapshot Isolation (MVCC)

> చాలా DBs (Postgres, Oracle) **MVCC** (Multi-Version Concurrency Control) వాడతాయి - ప్రతి transaction ఒక consistent snapshot చూస్తుంది. Readers writers ని block చేయరు (versions వల్ల). వేగం + isolation. కానీ **write skew** అనుమతిస్తుంది.

Key Points

- Isolation level = concurrency anomalies vs performance trade-off (serializable = safe కానీ slow)
- Serializability = transactions; Linearizability = single-object real-time
- MVCC (snapshot isolation) = readers/writers block చేయకుండా isolation (కానీ write skew)
- Money/inventory → serializable లేదా careful locking; reads → snapshot isolation సరిపోతుంది

---

41. Stream Processing (Batch vs Stream)

వివరణ

పెద్ద data ని process చేయడానికి 2 మార్గాలు: **Batch** (పోగుపడ్డ data ని periodically) vs **Stream** (వచ్చిన కొద్దీ realtime). SSE level analytics, real-time features కి stream processing కీలకం.

Batch vs Stream

	Batch	Stream
Data	Bounded (finite chunk)	Unbounded (continuous)
Latency	ఎక్కువ (గంటలు/రోజులు)	తక్కువ (seconds)

	Batch	Stream
ఉదా	Daily reports, ETL	Real-time dashboards, fraud detection
Tools	Spark, Hadoop MapReduce	Flink, Kafka Streams, Spark Streaming

BATCH VS STREAM PROCESSING

LAMBDA ARCHITECTURE – రెండూ కలిపి

Speed layer దాదాపు-సరైన సంఖ్యలని నిమిషాల్లో ఇస్తుంది (dashboard కి). Batch layer ఖచ్చితమైన సంఖ్యలని గంటకోసారి ఇస్తుంది (billing కి), మరియు speed layer ని దిద్దుతుంది. Raw events ని ఎప్పుడూ దాచుకుంటాం – bug fix చేశాక మళ్ళీ process చేయడానికి.

Windowing (streams లో సమయాన్ని విభజించడం)

> Unbounded stream ని process చేయాలంటే **windows** గా విభజించాలి ("గత 5 నిమిషాల్లో ఎన్ని clicks?").

Window రకం	వివరణ
Tumbling	Fixed, non-overlapping (0-5, 5-10, 10-15 min)
Sliding	Overlapping (0-5, 1-6, 2-7 min)
Session	Activity gap బట్టి (user idle అయ్యేదాకా)

Event Time vs Processing Time + Watermarks

> **సమస్య:** Event 10:00కి జరిగింది కానీ network delay వల్ల 10:05కి వచ్చింది (late/out-of-order). ఏ time వాడాలి?
>
> - **Event time** = నిజంగా జరిగిన సమయం (correct కానీ late data handle చేయాలి)
> - **Processing time** = system కి వచ్చిన సమయం (simple కానీ inaccurate)
> - **Watermark** = "ఈ timestamp కంటే పాత events ఇక రావు" అనే signal → window ని ఎప్పుడు close చేయాలో నిర్ణయిస్తుంది

Exactly-Once in Streams

> Stream processor crash అయి restart అయితే - events double process అవ్వకుండా. **Checkpointing** (state snapshot) + **idempotent sinks** + transactional writes (Kafka + Flink) → exactly-once.

Lambda vs Kappa Architecture

	Lambda	Kappa
ఏం	Batch layer + Speed layer (రెండూ)	Stream మాత్రమే (batch = stream replay)
Complexity	ఎక్కువ (2 codebases)	తక్కువ (ఒకటే)

Key Points

- Batch = high-latency, high-throughput; Stream = low-latency, continuous
- Windowing (tumbling/sliding/session) = unbounded stream ని process చేయడానికి
- Event time + watermarks = late/out-of-order data ని correct గా handle చేయడం
- Exactly-once = checkpointing + idempotent sinks (Flink, Kafka Streams)

---

42. Multi-Region & CRDTs

వివరణ

Global scale కి **multi-region** deployment (latency తగ్గించడం, disaster tolerance). కానీ regions మధ్య data ని ఎలా consistent గా ఉంచాలి? **Active-active** writes → conflicts → **CRDTs** పరిష్కారం.

Multi-Region Strategies

Strategy	వివరణ	Trade-off
Active-Passive	ఒక region writes, మిగతా read replicas	Simple కానీ failover latency
Active-Active	అన్ని regions writes accept	0 downtime కానీ conflict resolution అవసరం

Conflict Resolution

> రెండు regions ఒకే data ని ఏకకాలంలో మార్చితే - conflict. పరిష్కారాలు:
>
> - **Last-Write-Wins (LWW)** - timestamp ఎక్కువ ఉన్నది గెలుస్తుంది (simple కానీ data loss)
> - **Vector Clocks** - causality track చేసి conflicts detect
> - **CRDTs** - గణితపరంగా conflict-free (auto-merge)

CRDT (Conflict-free Replicated Data Type)

> **CRDT** = ఏ order లో merge చేసినా ఒకే ఫలితం వచ్చే data structures (commutative, associative, idempotent). Regions విడిగా update అయినా, merge అయ్యాక అందరూ **converge** అవుతారు - conflict లేకుండా.

Code - G-Counter + LWW-Register CRDT

```
// G-Counter (Grow-only Counter) - ప్రతి node కి విడి count, merge = per-node max
class GCounter {
  constructor(nodeId) {
    this.nodeId = nodeId;
    this.counts = {};
  }
  increment(n = 1) {
    this.counts[this.nodeId] = (this.counts[this.nodeId] || 0) + n;
  }
  value() {
    return Object.values(this.counts).reduce((a, b) => a + b, 0);
  }
  merge(other) {
    const merged = new GCounter(this.nodeId);
    const nodes = new Set([
      ...Object.keys(this.counts),
      ...Object.keys(other.counts),
    ]);
    for (const n of nodes)
      merged.counts[n] = Math.max(this.counts[n] || 0, other.counts[n] || 0);
    return merged;
  }
}

// రెండు regions network partition లో విడిగా increment
const usa = new GCounter("usa");
const india = new GCounter("india");
usa.increment(3); // USA: 3 likes
india.increment(5); // India: 5 likes (ఒకరికొకరు తెలియదు)
console.log("USA sees:", usa.value(), "| India sees:", india.value()); // 3 | 5

// Partition heal → merge (ఏ order లో అయినా ఒకటే ఫలితం - converge)
console.log("Merged:", usa.merge(india).value()); // 8 (conflict లేదు!)

// LWW-Register - timestamp ఎక్కువ ఉన్న write గెలుస్తుంది
class LWWRegister {
  constructor() {
    this.value = null;
    this.ts = 0;
  }
  set(value, ts) {
    if (ts > this.ts) {
      this.value = value;
      this.ts = ts;
    }
  }
  merge(other) {
    if (other.ts > this.ts) {
```

```
            this.value = other.value;
            this.ts = other.ts;
          }
        }
      }
      const r1 = new LWWRegister(),
        r2 = new LWWRegister();
      r1.set("blue", 100);
      r2.set("red", 105);
      r1.merge(r2);
      console.log("LWW winner:", r1.value); // red (higher timestamp గెలిచింది)
```

CRDT ఎక్కడ

- Collaborative editing (Google Docs, Figma), shopping carts (Amazon), likes/counters, presence
- Redis, Riak, Automerge, Yjs - CRDT support

Key Points

- Multi-region = latency↓ + DR; కానీ active-active → conflicts
- LWW (simple, data loss) < Vector clocks (detect) < CRDTs (auto-resolve)
- CRDT = commutative + associative + idempotent → ఏ order merge అయినా converge
- Data locality: user data ని వాళ్ళ region లో (GDPR + latency)

---

43. Security in System Design

వివరణ

SSE level లో security ని design లో భాగంగా చూపించాలి - authentication, authorization, encryption, secrets, attacks నివారణ.

Authentication vs Authorization

	Authentication (AuthN)	Authorization (AuthZ)
ప్రశ్న	"నువ్వు ఎవరు?"	"నీకు అనుమతి ఉందా?"
ఉదా	Login (password, OTP)	"ఈ file delete చేయగలవా?"
పద్ధతి	Session, JWT, OAuth	RBAC, ABAC, ACL

Tokens: Session vs JWT

	Session (server state)	JWT (stateless)
State	Server లో session store	Token లోనే (self-contained)

	Session (server state)	JWT (stateless)
Scale	Session store lookup	Stateless (verify signature)
Revoke	సులభం (delete session)	కష్టం (expiry దాకా valid)

> **JWT** = signed token (header.payload.signature). Server signature verify చేస్తే చాలు - DB lookup అక్కర్లేదు (stateless, scalable). కానీ revoke కష్టం → short expiry + refresh token.

OAuth 2.0 (delegation)

> "Google తో login" - నీ Google password ని 3rd party కి ఇవ్వకుండా, Google ఒక **token** ఇస్తుంది. OAuth = access delegation (password share చేయకుండా).

Encryption

రకం	ఎప్పుడు
In Transit	TLS/HTTPS (network మీద data)
At Rest	DB/disk encryption (stored data)
End-to-End	Sender-receiver మాత్రమే (WhatsApp - server కూడా చదవలేదు)

ముఖ్య Security Concerns

Attack/Concern	నివారణ
DDoS	Rate limiting, CDN, WAF
SQL Injection	Parameterized queries
Secrets	Vault/KMS (code లో hardcode వద్దు!)
Data privacy	GDPR (data residency, right to delete), PII encryption

Key Points

- AuthN (ఎవరు) vs AuthZ (అనుమతి) - రెండూ వేరు
- JWT = stateless/scalable కానీ revoke కష్టం; Session = revoke సులభం కానీ stateful
- Encrypt in-transit (TLS) + at-rest; sensitive → end-to-end
- Secrets ఎప్పుడూ Vault/KMS లో (code/config లో కాదు); rate limit + WAF for DDoS

44. Cost, Load Shedding & Graceful Degradation

వివరణ

SSE = system పనిచేస్తేనే కాదు, **cost-effective** గా, **overload లో కూడా** పనిచేయాలి. Infinite resources లేవు - trade-offs.

Load Shedding

> **Overload వచ్చినప్పుడు అన్నీ fail అవ్వకుండా, తక్కువ ప్రాధాన్య requests ని ఉద్దేశపూర్వకంగా drop చేయడం.** Hospital triage లాంటిది - అందరినీ కాపాడలేకపోతే, ముఖ్యమైన వాళ్ళను ముందు.

- Priority బట్టి drop (paid users > free; critical > analytics)
- 503 Service Unavailable + Retry-After

Graceful Degradation

> Overload/failure లో **పూర్తిగా down కాకుండా, తక్కువ features తో** పనిచేయడం.

> **ఉదా - Amazon:** Recommendation service down అయితే, page crash కాదు - recommendations లేకుండా (లేదా generic) చూపిస్తుంది. Core (buy) పనిచేస్తుంది.

Backpressure

> Downstream slow అయితే, upstream ని "నెమ్మదించు" అని signal (లేకపోతే queue పేలుతుంది, OOM). Reactive systems, streaming లో కీలకం.

Cost Optimization

Technique	ఎలా ఆదా
Tiered storage	Hot (SSD, ఖరీదు) vs Cold (S3 Glacier, చౌక)
Auto-scaling	Traffic బట్టి servers up/down (idle ఖర్చు తగ్గించడం)
Caching	DB/compute calls తగ్గించడం (cache చౌక)
Spot instances	Batch jobs కి cheap interruptible compute
Data retention	పాత data delete/archive (storage cost)

Key Points

- Load shedding = overload లో low-priority requests drop (అన్నీ fail కాకుండా)

- Graceful degradation = core features నిలబెట్టి, non-essential తగ్గించడం
- Backpressure = downstream slow అయితే upstream ని slow చేయడం (queue explosion నివారణ)
- Cost = tiered storage + auto-scaling + caching; SSE trade-offs లో cost కూడా భాగం

Part 8 – Advanced Case Studies

> SSE interviews లో అడిగే కష్టమైన, nuanced systems. వీటిలో correctness (money), realtime collaboration, latency (typeahead), reliability (scheduler) - deep trade-offs ఉంటాయి.

ఈ PART గురించి ఒక ముఖ్యమైన మాట

కింది case studies ఇక్కడ **సంక్షిప్తంగా** ఉన్నాయి – నేర్చుకున్న building blocks ని ఒక నిజమైన systemలో ఎలా కలుపుతారో చూపించడానికి.

ఇవే problems `HLD_Design_Problems_Telugu.pdf` lo చాలా లోతుగా ఉన్నాయి – ఒక్కో దానికి 6–11 పేజీలు: clarifying questions, పూర్తి capacity estimation, architecture diagrams, deep dive, failure analysis, మరియు **interview lo నోటితో చెప్పాల్సిన English script**.

ఎలా వాడాలి: ఈ document = *పరికరాల పెట్టె* (caching అంటే ఏమిటి, quorum అంటే ఏమిటి). ఆ book = *ఆ పరికరాలతో 18 నిజమైన problems ని ఎలా పరిష్కరించాలి*. ముందు ఇది, తర్వాత అది.

---

45. Payment System / Ledger

1. Requirements

- Money transfer (A → B); **exactly-once** (double-charge కూడదు); strong consistency; full audit trail
- **NFR:** Correctness > availability (డబ్బు తప్పు కూడదు) → CP system

2. Core: Double-Entry Ledger

> ప్రతి transaction = ఒక **debit** + ఒక **credit** (sum ఎప్పుడూ 0). Money create/destroy కాదు, కేవలం move. Ledger **append-only** (immutable) - audit + correctness.

Code

```
class Ledger {
  #entries = []; // append-only (immutable audit trail)
  #processed = new Set(); // idempotency keys

  transfer(idempotencyKey, from, to, amount) {
    // Idempotency - retry/duplicate అయినా ఒక్కసారే (Topic 18)
    if (this.#processed.has(idempotencyKey)) {
      console.log(`⏭️ Duplicate ${idempotencyKey} - skip`);
      return;
    }
    if (amount <= 0) throw new Error("Invalid amount");
```

```
    // Double-entry: from debit, to credit (atomic pair)
    this.#entries.push({ key: idempotencyKey, account: from, delta: -amount });
    this.#entries.push({ key: idempotencyKey, account: to, delta: +amount });
    this.#processed.add(idempotencyKey);
    console.log(`💸 ${from} → ${to}: ${amount}`);
  }

  balance(account) {
    return this.#entries
      .filter((e) => e.account === account)
      .reduce((s, e) => s + e.delta, 0);
  }
  totalDelta() {
    return this.#entries.reduce((s, e) => s + e.delta, 0);
  } // invariant: 0
}

const ledger = new Ledger();
ledger.transfer("seed", "world", "alice", 1000); // alice కి 1000 seed
ledger.transfer("txn1", "alice", "bob", 300);
ledger.transfer("txn1", "alice", "bob", 300); // retry - idempotent (double-charge
కాదు)

console.log("Alice:", ledger.balance("alice")); // 700
console.log("Bob:", ledger.balance("bob")); // 300
console.log("Total delta (must be 0):", ledger.totalDelta()); // 0 (money conserved)
```

3. Design Points

అంశం	పరిష్కారం
Exactly-once	Idempotency key (Topic 18) - duplicate skip
Consistency	Strong (ACID transaction, serializable isolation)
Audit	Append-only ledger (immutable, event sourcing - Topic 25)
Cross-service	Saga (Topic 19) - payment → inventory → shipping, compensate on fail
Reconciliation	Periodic - ledger balances match actual?

4. Deep Dive

- **CP over AP** - partition లో unavailable మేలు (wrong balance కంటే)
- **2-phase**: authorize (hold) → capture (charge) - card payments
- **Idempotency + audit + strong consistency** = payment core
- **External gateway** (Stripe) - webhook + idempotency + retry

> **SSE insight:** Payments = correctness మొదటిది. "Eventual consistency ఇక్కడ వద్దు - double spend కూడదు. Idempotency key + ACID + append-only ledger + reconciliation" అని ఎందుకో justify చేయి.

46. Collaborative Editing (Google Docs)

1. Requirements

- అనేక users ఒకే document ని ఏకకాలంలో edit; realtime sync; conflict-free; offline support
- Low latency (keystroke వెంటనే కనిపించాలి)

2. Core సమస్య: Concurrent Edits

> User A "cat" లో index 0కి "s" చేర్చాడు → "scat". అదే క్షణం User B index 3కి "!" చేర్చాడు → "cat!". రెండూ merge అయితే? Indices shift అవుతాయి → conflict. పరిష్కారం: **OT** లేదా **CRDT**.

OT vs CRDT

	OT (Operational Transform)	CRDT
ఎలా	Operations ని ఒకదాని బట్టి transform (index adjust)	Characters కి unique IDs, order గణితంగా
Server	Central server అవసరం (transform coordinate)	Peer-to-peer possible (server optional)
Complexity	Transform logic కష్టం	Metadata ఎక్కువ (ప్రతి char కి ID)
ఉదా	Google Docs (classic)	Figma, Yjs, Automerge

> **OT ఆలోచన:** Op B ని, ఇప్పటికే apply అయిన Op A బట్టి **transform** చేస్తారు (B యొక్క index ని adjust). అందరూ same final state కి converge.
>
> **CRDT ఆలోచన:** ప్రతి character కి unique, ordered ID (fractional index). Insert = రెండు IDs మధ్య కొత్త ID. Order గణితంగా నిర్ణయమవుతుంది → conflict లేదు (Topic 42).

3. Architecture

```
User A ─┐
        ├─ WebSocket ─→ Collab Server (OT transform / CRDT merge) ─→ persist (DB)
User B ─┘                    ↓ broadcast merged ops కి అందరికీ
Offline edits → reconnect → sync (CRDT auto-merge)
```

4. Deep Dive

- **Realtime** → WebSocket (Topic 24), ops broadcast
- **Conflict-free** → OT (server-coordinated) లేదా CRDT (Topic 42, offline-friendly)
- **Cursor presence** → ఇతరుల cursors చూపించడం (ephemeral, CRDT-like)
- **History/undo** → ops log (event sourcing - Topic 25)

- **Persistence** → periodic snapshots + op log (పూర్తి replay ఖరీదు)

> **SSE insight:** "Last-write-wins వద్దు - keystrokes పోతాయి. OT లేదా CRDT" అని చెప్పి, offline support కావాలంటే CRDT (Topic 42) ఎందుకు మంచిదో explain చేయి.

---

47. Typeahead / Autocomplete

1. Requirements

- User type చేస్తుంటే top suggestions (prefix match); **cực fast** (<100ms, ప్రతి keystroke); popularity ranked
- Massive scale (Google search box)

2. Core: Trie (Prefix Tree)

> **Trie** = prefix ఆధారంగా వేగవంతమైన lookup. ప్రతి node ఒక character; root నుండి path = ఒక word. Prefix కి navigate చేసి, ఆ subtree లోని words = suggestions.

Code

```
class TrieNode {
  constructor() {
    this.children = {};
    this.isEnd = false;
    this.freq = 0;
  }
}
class Autocomplete {
  #root = new TrieNode();
  insert(word, freq = 1) {
    let node = this.#root;
    for (const ch of word) {
      if (!node.children[ch]) node.children[ch] = new TrieNode();
      node = node.children[ch];
    }
    node.isEnd = true;
    node.freq = freq;
  }
  // prefix తో మొదలయ్యే top words (popularity క్రమంలో)
  suggest(prefix, limit = 5) {
    let node = this.#root;
    for (const ch of prefix) {
      if (!node.children[ch]) return []; // prefix లేదు
      node = node.children[ch];
    }
    const results = [];
    const dfs = (n, path) => {
      if (n.isEnd) results.push({ word: prefix + path, freq: n.freq });
```

```
      for (const ch in n.children) dfs(n.children[ch], path + ch);
    };
    dfs(node, "");
    return results
      .sort((a, b) => b.freq - a.freq)
      .slice(0, limit)
      .map((r) => r.word);
  }
}

const ac = new Autocomplete();
[
  ["telugu", 100],
  ["telegram", 80],
  ["telephone", 60],
  ["television", 90],
  ["system", 50],
].forEach(([w, f]) => ac.insert(w, f));

console.log(ac.suggest("tel")); // [telugu, television, telegram, telephone] (freq
order)
console.log(ac.suggest("sys")); // [ 'system' ]
console.log(ac.suggest("xyz")); // [] (match లేదు)
```

3. Scale కి Design Points

అంశం	పరిష్కారం
Latency	Trie in-memory; edge/CDN దగ్గర cache
Popularity	ప్రతి node లో top-K precomputed (DFS ప్రతిసారి ఖరీదు)
Scale	Trie ని shard (prefix బట్టి), replicate
Updates	Search logs → batch job → Trie rebuild (realtime కాదు)
Personalization	User history + global blend

4. Deep Dive

- **Precompute top-K per node** → runtime DFS లేకుండా O(prefix length)
- **Debounce** client-side (ప్రతి keystroke కి కాదు)
- **Batch update** - popularity ని offline లెక్కించి Trie rebuild
- **Fuzzy match** (typos) → edit distance (ఖరీదు, optional)

> **SSE insight:** "Trie వేగం, కానీ ప్రతి node కి **top-K precompute** చేస్తే runtime DFS అవసరం లేదు - <10ms. Updates realtime కాదు, batch (search logs → rebuild)" - latency vs freshness trade-off చూపించు.

48. Distributed Job Scheduler (Cron at Scale)

1. Requirements

- Millions scheduled jobs (once / recurring); reliable (miss కూడదు); at-least-once execution; scale
- ఉదా: reminders, report generation, retries

2. Core: ఎవరు, ఎప్పుడు run చేయాలి?

> Jobs ని time-ordered store చేసి, "ఇప్పుడు due అయినవి" pick చేసి workers కి పంచడం. Single scheduler = SPOF + bottleneck → distributed.

3. Architecture

```
Job Submit → DB (jobs, nextRunTime indexed)
Scheduler (leader-elected) → due jobs poll → Queue (Kafka) → Worker pool → execute
                                                               ↓ fail → retry
(backoff) → DLQ
Leader election (Topic 20: Zookeeper/etcd) → ఒక్క scheduler active (duplicate కాకుండా)
```

4. Design Points

అంశం	పరిష్కారం
Reliability	Job DB persist; worker crash → queue లో ఉంటుంది, retry
No duplicate	Leader election (ఒక్క scheduler) + idempotent jobs (Topic 18)
Scale	Jobs ని shard (time buckets); workers horizontal
Precision	Poll interval trade-off (వేగం vs DB load); time-wheel
Recurring	Run అయ్యాక nextRunTime update (reschedule)
At-least-once	Retry + idempotency (exactly-once ఖరీదు)

5. Deep Dive

- **Leader election** (Topic 20) → duplicate scheduling నివారణ
- **Time-based sharding** → ఒక్కో scheduler ఒక్కో time-bucket
- **Hot shard** (ఒకే second కి million jobs) → jitter/spread
- **Missed jobs** (scheduler down) → recovery on restart (past-due jobs scan)
- **Idempotency** → job double-run అయినా safe (at-least-once accept)

> **SSE insight:** "Single cron = SPOF. Leader election (duplicate నివారణ) + queue (reliability) + idempotent workers (at-least-once safe) + time-sharding (scale)" - reliability + scale రెండూ address చేయి.

Part 9 – Interview & Reference

49. SDE2 vs SSE - ఏం Expect చేస్తారు

వివరణ

ఒకే system design ప్రశ్న ఇచ్చినా, SDE2 మరియు SSE నుండి వేర్వేరు లోతు ఆశిస్తారు. తేడా తెలిస్తే సరిగ్గా aim చేయవచ్చు.

ఏం చూస్తారు

అంశం	SDE2	SSE (Senior)
Requirements	ఇచ్చినవి handle చేయడం	Ambiguity ని clarify + scope చేయడం
Building blocks	సరిగ్గా వాడటం (LB, cache, DB)	**ఎందుకు** ఈ block, alternatives ఏంటి
Trade-offs	తెలుసు, ప్రస్తావిస్తాడు	ప్రతి నిర్ణయాన్ని justify + defend చేస్తాడు
Scale	Millions	100M+, bottlenecks, multi-region
Failure modes	ప్రాథమికంగా	Deep (partition, cascading, data loss)
Estimation	చేయగలడు	Estimation తో decisions drive చేస్తాడు
Deep dive	ఒక component	అనేక, deep, edge cases
Operations	తెలుసు	Monitoring, DR, deployment - ownership
Communication	స్పష్టం	Interviewer ని lead చేస్తాడు, drive చేస్తాడు

SSE ని ప్రత్యేకంగా చూపించే signals

- **"ఎందుకు" ప్రతిదానికీ** - "SQL వాడాను ఎందుకంటే transactions... NoSQL కూడా option కానీ joins కావాలి కాబట్టి SQL"
- **Failure గురించి proactive** - "ఈ service down అయితే...", "network partition అయితే..."
- **Numbers తో reasoning** - "230K read QPS కాబట్టి ఒక DB సరిపోదు, cache + replicas అవసరం"
- **Trade-offs స్పష్టంగా** - "ఇది latency తగ్గిస్తుంది కానీ consistency తగ్గుతుంది - ఈ feature కి అది ఓకే"
- **Bottleneck identification** - "ఇక్కడ bottleneck ఇది, దీన్ని ఇలా scale చేస్తా"

> **గుర్తుంచుకో:** SDE2 = **correct** design. SSE = correct + **justified** + **resilient** + **operable** design. Depth మరియు reasoning = seniority signal.

50. HLD Interview Framework (Step-by-step)

వివరణ

Topic 2 లో చూసిన framework ని ఒక reference checklist గా. "Design X" అడిగినప్పుడు ఈ క్రమం.

The Checklist (ఒక్కో అడుగు)

```
□ 1. REQUIREMENTS (5 min)
     - Functional: ఏ features? (MVP scope narrow చేయి)
     - Non-functional: scale? consistency? latency? read/write ratio?
     - "ఇవి assumptions - confirm చేయండి"

□ 2. ESTIMATION (5 min)
     - DAU, QPS (read + write), storage/year, bandwidth
     - ఈ numbers ఏ decisions drive చేస్తాయో చెప్పు

□ 3. API DESIGN (3-5 min)
     - ముఖ్య endpoints (REST/gRPC), request/response

□ 4. DATA MODEL (5 min)
     - Entities, SQL vs NoSQL (+ ఎందుకు), schema, shard key

□ 5. HIGH-LEVEL DESIGN (10 min)
     - Boxes: Client → LB → Services → Cache → DB → Queue
     - Data flow గీయి, మాట్లాడుతూ

□ 6. DEEP DIVE (15 min)
     - Bottlenecks: ఎక్కడ break అవుతుంది?
     - Scaling: sharding, caching, replication
     - 1-2 components deep (interviewer choice)

□ 7. WRAP UP (5 min)
     - Bottlenecks summary, trade-offs
     - Failure modes, monitoring, future improvements
```

సమయం లేకపోతే priority

> Requirements → Estimation → High-level diagram → ఒక deep dive. మిగతావి skip చేసినా ఈ 4 core.

Key Points

- ఎప్పుడూ requirements + estimation తో మొదలుపెట్టు (నేరుగా boxes కాదు)
- Simple design → "ఇది scale అవ్వదు ఎందుకంటే..." → evolve చేయి
- Think aloud, trade-offs, numbers - ప్రతి అడుగులో

---

51. Numbers & Latency Cheat Sheet

Latency Numbers (గుర్తుంచుకో)

Operation	**Time (approx)**
L1 cache reference	~1 ns
Main memory (RAM) reference	~100 ns
SSD random read	~100 μs (0.1 ms)
Network round-trip (same datacenter)	~0.5 ms
HDD seek	~10 ms
Network round-trip (cross-continent)	~150 ms

> **ముఖ్య పాఠం:** RAM ≫ SSD ≫ Disk ≫ Network(cross-region). అందుకే **cache (RAM)** వేగం; cross-region calls ఖరీదు.

Capacity Numbers

అంశం	**విలువ**
సెకన్లు/రోజు	~86,400 (~10^5)
1M requests/day	~12 QPS
1B requests/day	~11,600 QPS
Char (ASCII)	1 byte
Powers: KB→MB→GB→TB	×1000 ఒక్కోటి

Powers of Two (storage)

Power	**Value**	**పేరు**
2^10	~1,000	KB
2^20	~1 million	MB
2^30	~1 billion	GB
2^40	~1 trillion	TB

Availability Downtime

Availability	Downtime/year
99%	3.65 రోజులు
99.9%	8.75 గంటలు
99.99%	52 నిమిషాలు
99.999%	5 నిమిషాలు

---

52. Memory Tips + Common Mistakes

Building Blocks (గుర్తుంచుకునే analogy)

Concept	గుర్తుంచుకోవడానికి
Load Balancer	Bank queue manager (counter పంచడం)
Cache	Fridge (market కి వెళ్ళకుండా)
CDN	ప్రతి నగరంలో Amazon warehouse
Sharding	Library branches (A-M, N-Z)
Replication	Xerox copies (backup + read scale)
Consistent Hashing	గుండ్రటి బల్ల (కొత్తవాడు వస్తే కొంతే మారుతుంది)
Message Queue	Restaurant order slips
CAP	Phone line తెగితే - consistent లేదా available
Circuit Breaker	Electrical fuse (fail fast)
Bloom Filter	"ఖచ్చితంగా లేదు" లేదా "బహుశా ఉంది"
Idempotency	Lift button (ఎన్నిసార్లు నొక్కినా ఒక్కటే)
Saga	Compensating steps (వెనక్కి undo)

SSE Deep Dives (గుర్తుంచుకునే analogy)

Concept	గుర్తుంచుకోవడానికి
LSM-tree	Memtable (memory) → SSTable (disk flush); write-fast
B-tree	In-place sorted; read-fast (SQL)

Concept	గుర్తుంచుకోవడానికి
Serializability	Transactions ఏదో ఒక serial order
Linearizability	Single object, real-time order
CRDT	ఏ order merge అయినా converge (conflict-free)
Watermark	"ఈ time కంటే పాత events ఇక రావు"
Load Shedding	Hospital triage (ముఖ్యమైనవి ముందు)
Graceful Degradation	Recommendations పోయినా "buy" పనిచేస్తుంది
Double-entry Ledger	ప్రతి debit కి credit (sum = 0)
Trie	Prefix tree (typeahead autocomplete)

Common Mistakes (వీటిని చేయకు)

❌ తప్పు	✅ సరైనది
నేరుగా boxes గీయడం	ముందు requirements + estimation
Silent గా ఆలోచించడం	Think aloud
"Microservices always"	Monolith first, అవసరమైతే విడగొట్టు
ఒకే perfect solution	Trade-offs మాట్లాడు
"NoSQL = scale" గుడ్డిగా	Access pattern బట్టి నిర్ణయించు
Numbers లేకుండా decisions	Estimation తో justify
Failure modes విస్మరించడం	"ఇది fail అయితే?" proactive
Over-engineering (అన్ని patterns)	అవసరమైనవి మాత్రమే (YAGNI)
Single point of failure వదిలేయడం	Redundancy ప్రతి layer లో

ఒక్క వాక్యంలో ముఖ్య సూత్రాలు

- **Scale:** Stateless + horizontal + cache + shard
- **Availability:** Redundancy + failover (SPOF తీసేయి)
- **Read-heavy:** Cache + replicas
- **Write-heavy:** Sharding + queue (async)
- **Consistency:** Money → strong; social → eventual
- **Everything is a trade-off** - free lunch లేదు

ముగింపు

> **గుర్తుంచుకో:**
>
> - HLD = **scale + trade-offs** యొక్క craft. ప్రతి box కి "ఎందుకు?" సమాధానం ఉండాలి.
> - Requirements → Estimation → Design → Deep Dive → Trade-offs. ఈ క్రమం మర్చిపోకు.
> - SDE2 = correct design; **SSE = correct + justified + resilient + operable**.
> - "Perfect design" లేదు - *justified* design ఉంది. Numbers తో reasoning = seniority.

ఈ 3 documents కలిస్తే - **OOP (basics)** → **LLD (class design)** → **HLD (system design)** - ఒక complete తెలుగు engineering reference:

- `OOPS_Telugu.md` - Object-Oriented Programming
- `LLD_Telugu.md` - Low-Level Design (patterns)
- `HLD_Telugu.md` - High-Level Design (system design)

JavaScript/System Design HLD - Complete Telugu Guide (SDE2 & SSE) Runnable code snippets Node.js లో run చేసి verify చేయబడ్డాయి ✅